

ГЛАВА 10

ОБОБЩЕННЫЕ АЛГОРИТМЫ

В этой главе...

10.1. Краткий обзор	482
10.2. Первый взгляд на алгоритмы	485
10.3. Перенастройка функций	494
10.4. Возвращаясь к итераторам	514
10.5. Структура обобщенных алгоритмов	525
10.6. Алгоритмы, специфические для контейнеров	531
Резюме	534
Термины	534

В библиотечных контейнерах определен на удивление небольшой набор функций. Вместо того чтобы снабжать каждый контейнер большим количеством одинаковых функций, библиотека предоставляет набор алгоритмов, большинство из которых не зависит от конкретного типа контейнера. Эти алгоритмы называют *обобщенными* (*generic*), поскольку применимы они и к контейнерам разных типов, и к разным типам элементов.

Темой данной главы являются не только обобщенные алгоритмы, но и более подробное изучение итераторов.

В последовательных контейнерах определено немного операций: по большей части они позволяют добавлять и удалять элементы, обращаться к первому или последнему элементу, определять, не пуст ли контейнер, и получать итераторы на первый элемент или следующий после последнего.

Но может понадобиться и множество других вспомогательных операций: поиск определенного элемента, замена или удаление некоего значения, перепорядочивание элементов контейнера и т.д.

Чтобы не создавать каждую из этих функций как член контейнера каждого типа, стандартная библиотека определяет набор *обобщенных алгоритмов* (*generic algorithm*). *Алгоритмами* они называются потому, что реализуют классические алгоритмы, такие как сортировка и поиск, а *обобщенными* — потому, что работают с контейнерами *любых* типов, включая массивы встроенных ти-

пов, и, как будет продемонстрировано далее, с последовательностями других видов, а не только с такими библиотечными типами, как `vector` или `list`.



10.1. Краткий обзор

Большинство алгоритмов определено в заголовке `algorithm`. Библиотека определяет также набор обобщенных числовых алгоритмов в заголовке `numeric`.

Обычно алгоритмы воздействуют не на сам контейнер, а работают, перебирая диапазон элементов, заданный двумя итераторами (см. раздел 9.2.1, стр. 426). Перебирая диапазон, алгоритм, как правило, выполняет некое действие с каждым элементом. Предположим, например, что имеется вектор целых чисел и необходимо узнать, содержит ли этот вектор некое значение. Проще всего ответить на этот вопрос, воспользовавшись библиотечным алгоритмом `find()`:

```
int val = 42; // искомое значение
// result обозначит искомый элемент, если он есть в векторе,
// или значение vec.cend(), если нет
auto result = find(vec.cbegin(), vec.cend(), val);
// отображение результата
cout << "The value " << val
      << (result == vec.cend()
          ? " is not present" : " is present") << endl;
```

Первые два аргумента функции `find()` являются итераторами, обозначающими диапазон элементов, а третий аргумент — это значение. Функция `find()` сравнивает каждый элемент указанного диапазона с заданным значением и возвращает итератор на первый элемент, соответствующий этому значению. При отсутствии соответствия функция `find()` возвращает свой второй итератор, означая неудачу поиска. Так, сравнив возвращаемое значение со вторым аргументом, можно определить, был ли найден элемент. Для проверки, свидетельствующей об успехе поиска значения, в операторе вывода используется условный оператор (см. раздел 4.7 стр. 208).

Поскольку функция `find()` работает с итераторами, ее можно использовать для поиска значения в любом контейнере. Рассмотрим пример применения функции `find()` для поиска значения в списке строк:

```
string val = "a value"; // искомое значение
// вызов, позволяющий найти строковый элемент в списке
auto result = find(lst.cbegin(), lst.cend(), val);
```

Аналогично, поскольку указатели действуют как итераторы встроенных массивов, функцию `find()` можно использовать для поиска в массиве:

```
int ia[] = {27, 210, 12, 47, 109, 83};  
int val = 83;  
int* result = find(begin(ia), end(ia), val);
```

Здесь для передачи указателей на первый и следующий после последнего элементы массива `ia` используются библиотечные функции `begin()` и `end()` (см. раздел 3.5.3, стр. 168).

Искать также можно в диапазоне, заданном переданными итераторами (или указателями), на его первый и следующий после последнего элементы. Например, следующий вызов ищет соответствие в элементах `ia[1]`, `ia[2]` и `ia[3]`:

```
// искать среди элементов, начиная с ia[1] и до, но не включая, ia[4]  
auto result = find(ia + 1, ia + 4, val);
```

Как работают алгоритмы

Для того чтобы изучить применение алгоритмов к контейнерам различных типов, немного подробнее рассмотрим функцию `find()`. Ее задачей является поиск указанного элемента в не отсортированной коллекции элементов. Концептуально функция `find()` должна выполнить следующие действия.

1. Обратиться к первому элементу последовательности.
2. Сравнить этот элемент с искомым значением.
3. Если элемент соответствует искомому, функция `find()` возвращает значение, идентифицирующее этот элемент.
4. В противном случае функция `find()` переходит к следующему элементу и повторяет этапы 2 и 3.
5. По достижении конца последовательности функция `find()` должна остановиться.
6. Достигнув конца последовательности, функция `find()` должна вернуть значение, означающее неудачу поиска. Тип этого значения должен быть совместимым с типом значения, возвращенного на этапе 3.

Ни одно из этих действий не зависит от типа контейнера, который содержит элементы. Пока есть итераторы, применимые для доступа к элементам, функция `find()` в любом случае не зависит от типа контейнера (или даже хранимых в контейнере элементов).

Итераторы делают алгоритмы независимыми от типа контейнера...

Все этапы работы функции `find()`, кроме второго, могут быть выполнены средствами итератора: оператор обращения к значению итератора предоставляет доступ к значению элемента; если элемент соответствует искомому, функция `find()` может вернуть итератор на этот элемент; оператор инкремента итератора переводит его на следующий элемент; итератор после

конца будет означать достижение функцией `find()` конца последовательности; функция `find()` вполне может вернуть итератор после конца (см. раздел 9.2.1, стр. 426), чтобы указать на неудачу поиска.

...но алгоритмы зависят от типа элементов

Хотя итераторы делают алгоритмы независимыми от контейнеров, большинство алгоритмов используют одну (или больше) функцию типа элемента. Например, этап 2 использует оператор `==` типа элемента для сравнения каждого элемента с предоставленным значением.

Другие алгоритмы требуют, чтобы тип элемента имел оператор `<`. Но, как будет продемонстрировано, большинство алгоритмов позволяют предоставить собственную функцию для использования вместо оператора, заданного по умолчанию.

Упражнения раздела 10.1

Упражнение 10.1. В заголовке `algorithm` определена функция `count()`, подобная функции `find()`. Она получает два итератора и значение, а возвращает количество обнаруженных в диапазоне элементов, обладающих искомым значением. Организуйте чтение в вектор последовательности целых чисел. Осуществите подсчет элементов с указанным значением.

Упражнение 10.2. Повторите предыдущую программу, но чтение значений организуйте в список (`list`) строк.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. АЛГОРИТМЫ НИКОГДА НЕ ИСПОЛЬЗУЮТ ФУНКЦИИ КОНТЕЙНЕРОВ

Общие алгоритмы никогда не используют функции контейнеров. Они работают исключительно с итераторами. Тот факт, что алгоритмы оперируют итераторами, а не функциями контейнера, возможно, удивителен, но он имеет глубокий смысл: когда используются “обычные” итераторы, алгоритмы не способны изменить размер исходного контейнера. Как будет продемонстрировано далее, алгоритмы способны изменять значения хранимых в контейнере элементов и перемещать их в контейнер. Однако они не способны ни добавлять, ни удалять сами элементы.

Как будет продемонстрировано в разделе 10.4.1 (стр. 514), существуют специальные классы итераторов, которые способны на несколько большее, чем просто перебор элементов. Они позволяют выполнять операции вставки. Когда алгоритм работает с одним из таких *итераторов*, возможен побочный эффект добавления элемента в контейнер, однако в самих *алгоритмах* это никогда не используется.



10.2. Первый взгляд на алгоритмы

Библиотека предоставляет больше ста алгоритмов. К счастью, у алгоритмов, как и у контейнеров, единообразная архитектура. Понимание этой архитектуры упростит изучение и использование всех ста с лишним алгоритмов без необходимости изучать каждый из них. В этой главе рассматривается использование алгоритмов и описаны характеризующие их общие принципы. В приложении А перечислены все алгоритмы согласно принципам их работы.

За небольшим исключением, все алгоритмы работают с диапазоном элементов. Далее этот диапазон мы будем называть *исходным диапазоном* (input range). Алгоритмы, работающие с исходным диапазоном, всегда получают его в виде двух первых параметров. Эти параметры являются итераторами, используемыми для обозначения первого и следующего после последнего элемента, подлежащих обработке.

Несмотря на то что большинство алгоритмов работают с одинаково обозначенным исходным диапазоном, они отличаются тем, как используются элементы этого диапазона. Проще всего подразделить алгоритмы на читающие, записывающие и меняющие порядок элементов.



10.2.1. Алгоритмы только для чтения

Много алгоритмов только читают значения элементов в исходном диапазоне, но никогда не записывают их. Функция `find()` и функция `count()`, использованная в упражнениях раздела 10.1 (стр. 484), являются примерами таких алгоритмов.

Другим предназначенным только для чтения алгоритмом является `accumulate()`, который определен в заголовке `numeric`. Функция `accumulate()` получает три аргумента. Первые два определяют диапазон суммируемых элементов, а третий — исходное значение для суммы. Предположим, что `vec` — это последовательность целых чисел.

```
// суммирует элементы вектора vec, начиная со значения 0  
int sum = accumulate(vec.cbegin(), vec.cend(), 0);
```

Приведенный выше код суммирует значения элементов вектора `vec`, используя 0 как начальное значение суммы.



Тип третьего аргумента функции `accumulate()` определяет, какой именно оператор суммы будет использован и каков будет тип возвращаемого значения функции `accumulate()`.

Алгоритмы и типы элементов

У того факта, что функция `accumulate()` использует свой третий аргумент как отправную точку для суммирования, есть важное последствие: он позволяет добавить тип элемента к типу суммы. Таким образом, тип элементов последовательности должен соответствовать или быть приводим к типу третьего аргумента. В этом примере элементами вектора `vec` могли бы быть целые числа, или числа типа `double`, или `long long`, или любого другого типа, который может быть добавлен к значению типа `int`.

Например, поскольку тип `string` имеет оператор `+`, функцию `accumulate()` можно использовать для конкатенации элементов вектора строк:

```
string sum = accumulate(v.cbegin(), v.cend(), string(""));
```

Этот вызов добавляет каждый элемент вектора `v` к первоначально пустой строке `sum`. Обратите внимание: третий параметр здесь явно указан как объект класса `string`. Передача строки как символьного литерала привела бы к ошибке при компиляции.

```
// ошибка: no + on const char*
string sum = accumulate(v.cbegin(), v.cend(), "");
```

Если бы был передан строковый литерал, типом суммируемых значений оказался бы `const char*`. Этот тип и определяет используемый оператор `+`. Поскольку тип `const char*` не имеет оператора `+`, этот вызов не будет компилироваться.



С алгоритмами, которые читают, но не пишут в элементы, обычно лучше использовать функции `cbegin()` и `cend()` (см. раздел 9.2.3, стр. 430). Но если возвращенный алгоритмом итератор планируется использовать для изменения значения элемента, то следует использовать функции `begin()` и `end()`.



Алгоритмы, работающие с двумя последовательностями

Еще один алгоритм только для чтения, `equal()`, позволяет определять, содержат ли две последовательности одинаковые значения. Он сравнивает каждый элемент первой последовательности с соответствующим элементом второй. Алгоритм возвращает значение `true`, если соответствующие элементы равны, и значение `false` в противном случае. Он получает три итератора: первые два (как обычно) обозначают диапазон элементов первой последовательности, а третий — первый элемент второй последовательности:

```
// roster2 должен иметь по крайней мере столько же элементов,
// сколько и roster1
equal(roster1.cbegin(), roster1.cend(), roster2.cbegin());
```

Поскольку функция `equal()` работает с итераторами, ее можно вызвать для сравнения элементов контейнеров разных типов. Кроме того, типы элементов также не обязаны совпадать, пока можно использовать оператор `==` для их сравнения. Например, контейнер `roster1` мог быть вектором `vector<string>`, а контейнер `roster2` — списком `list<const char*>`.

Однако алгоритм `equal()` делает критически важное предположение: подразумевает, что вторая последовательность по крайней мере не меньше первой. Этот алгоритм просматривает каждый элемент первой последовательности и подразумевает, что для него есть соответствующий элемент во второй последовательности.



Алгоритмы, получающие один итератор, обозначающий вторую последовательность, *подразумевают*, что вторая последовательность по крайней мере не меньше первой.

Упражнения раздела 10.2.1

Упражнение 10.3. Примените функцию `accumulate()` для суммирования элементов вектора `vector<int>`.

Упражнение 10.4. Если вектор `v` имеет тип `vector<double>`, в чем состоит ошибка вызова `accumulate(v.cbegin(), v.cend(), 0)` (если она есть)?

Упражнение 10.5. Что произойдет, если в вызове функции `equal()` для списков оба из них будут содержать строки в стиле C, а не библиотечные строки?

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ИТЕРАТОРЫ, ПЕРЕДАВАЕМЫЕ В КАЧЕСТВЕ АРГУМЕНТОВ

Некоторые алгоритмы читают элементы из двух последовательностей. Составляющие эти последовательности элементы могут храниться в контейнерах различных видов. Например, первая последовательность могла бы быть вектором (`vector`), а вторая списком (`list`), двухсторонней очередью (`deque`), встроенным массивом или другой последовательностью. Кроме того, типы элементов этих последовательностей не обязаны совпадать точно. Обязательно необходима возможность сравнивать элементы этих двух последовательностей. Например, в алгоритме `equal()` типы элемента не обязаны быть идентичными, на самом деле нужна возможность использовать оператор `==` для сравнения элементов этих двух последовательностей.

Алгоритмы, работающие с двумя последовательностями, отличаются способом передачи второй последовательности. Некоторые алгоритмы, такие как `equal()`, получают три итератора: первые два обозначают диапазон

первой последовательности, а третий — обозначает первый элемент во второй последовательности. Другие получают четыре итератора: первые два обозначают диапазон элементов в первой последовательности, а вторые два — диапазон второй последовательности.

Алгоритмы, использующие для обозначения второй последовательности один итератор, *подразумевают*, что вторая последовательность по крайней мере не меньше первой. Разработчик должен сам позаботиться о том, чтобы алгоритм не пытался обратиться к несуществующим элементам во второй последовательности. Например, алгоритм `equal()` сравнивает каждый элемент первой последовательности с соответствующим элементом второй. Если вторая последовательность является подмножеством первой, то возникает серьезная ошибка — функция `equal()` попытается обратиться к элементам после конца второй последовательности.



10.2.2. Алгоритмы, записывающие элементы контейнера

Некоторые алгоритмы способны записывать значения в элементы. Используя такие алгоритмы, следует соблюдать осторожность и предварительно удостовериться, что количество элементов, в которые алгоритм собирается внести изменения, по крайней мере не превосходит количество существующих элементов. Помните, что алгоритмы работают не с контейнерами, поэтому у них нет возможности самостоятельно изменить размер контейнера.

Некоторые алгоритмы осуществляют запись непосредственно в исходную последовательность. Эти алгоритмы в принципе безопасны: они способны переписать лишь столько элементов, сколько находится в указанном исходном диапазоне.

В качестве примера рассмотрим алгоритм `fill()`, получающий два итератора, обозначающих диапазон, и третий аргумент, являющийся значением. Функция `fill()` присваивает данное значение каждому элементу исходной последовательности:

```
fill(vec.begin(), vec.end(), 0); // обнулить каждый элемент
// присвоить половине последовательности значение 10
fill(vec.begin(), vec.begin() + vec.size()/2, 10);
```

Поскольку функция `fill()` пишет в переданную ей исходную последовательность до тех пор, пока она не закончится, запись вполне безопасна.



Алгоритмы не проверяют операции записи

Некоторые алгоритмы получают итератор, обозначающий конкретное значение. Эти алгоритмы присваивают новые значения элементам последовательности, начиная с элемента, обозначенного итератором назначения. На-

пример, функция `fill_n()` получает один итератор, количество и значение. Она присваивает предоставленное значение заданному количеству элементов, начиная с элемента, обозначенного итератором. Функцию `fill_n()` можно использовать для присвоения нового значения элементам вектора:

```
vector<int> vec; // пустой вектор
// используя вектор vec, предоставить ему разные значения
fill_n(vec.begin(), vec.size(), 0); // обнулить каждый элемент vec
```

Функция `fill_n()` подразумевала, что безопасно запишет указанное количество элементов. Таким образом, следующий вызов функции `fill_n()` подразумевает, что `dest` указывает на существующий элемент и что в последовательности есть по крайней мере `n` элементов, начиная с элемента `dest`.

```
fill_n(dest, n, val)
```

Это вполне обычная ошибка для новичка: вызов функции `fill_n()` (или подобного алгоритма записи элементов) для контейнера без элементов:

```
vector<int> vec; // пустой вектор
// катастрофа: попытка записи в 10 несуществующих элементов
// вектора vec
fill_n(vec.begin(), 10, 0);
```

Этот вызов функции `fill_n()` ведет к катастрофе. Должно быть записано десять элементов, но вектор `vec` пуст, и никаких элементов в нем нет. Результат непредсказуем, но, вероятнее всего, произойдет серьезный отказ во время выполнения.



Алгоритмы, осуществляющие запись по итератору назначения, подразумевают, что контейнер достаточно велик для содержания всех записываемых элементов.

Функция `back_inserter()`

Один из способов проверки, имеет ли контейнер достаточно элементов для записи, подразумевает использование *итератора вставки* (`insert iterator`), который позволяет *добавлять* элементы в базовый контейнер. Как правило, при присвоении значения элементу контейнера при помощи итератора осуществляется присвоение тому элементу, на который указывает итератор. При присвоении с использованием итератора вставки в контейнер добавляется новый элемент, равный правому значению.

Более подробная информация об итераторе вставки приведена в разделе 10.4.1 (стр. 514). Однако для иллюстрации безопасного применения алгоритмов, записывающих данные в контейнер, используем функцию `back_inserter()`, определенную в заголовке `iterator`.

Функция `back_inserter()` получает ссылку на контейнер и возвращает итератор вставки, связанный с данным контейнером. Попытка присвоения значения элементу при помощи этого итератора приводит к вызову функции `push_back()`, добавляющей элемент с данным значением в контейнер.

```
vector<int> vec; // пустой вектор
auto it = back_inserter(vec); // присвоение при помощи it добавляет
                               // элементы в vec
*it = 42;           // теперь vec содержит один элемент со значением 42
```

Функцию `back_inserter()` зачастую применяют для создания итератора, используемого в качестве итератора назначения алгоритмов. Рассмотрим пример:

```
vector<int> vec; // пустой вектор
// ok: функция back_inserter() создает итератор вставки,
// который добавляет элементы в вектор vec
fill_n(back_inserter(vec), 10, 0); // добавляет 10 элементов в vec
```

На каждой итерации функция `fill_n()` присваивает элемент заданной последовательности. Поскольку ей передается итератор, возвращенный функцией `back_inserter()`, каждое присвоение вызовет функцию `push_back()` вектора `vec`. В результате этот вызов функции `fill_n()` добавит в конец вектора `vec` десять элементов, каждый со значением 0.

Алгоритм `copy()`

Алгоритм `copy()` — это еще один пример алгоритма, записывающего элементы последовательности вывода, обозначенной итератором назначения. Этот алгоритм получает три итератора. Первые два обозначают исходный диапазон, а третий — начало последовательности вывода. Этот алгоритм копирует элементы из исходного диапазона в элементы вывода. Важно, чтобы переданный функции `copy()` контейнер вывода был не меньше исходного диапазона.

В качестве примера используем функцию `copy()` для копирования одного встроеного массива в другой:

```
int a1[] = {0,1,2,3,4,5,6,7,8,9};
int a2[sizeof(a1)/sizeof(*a1)]; // a2 имеет тот же размер, что и a1
// указывает на следующий элемент после последнего скопированного в a2
auto ret = copy(begin(a1), end(a1), a2); // копирует a1 в a2
```

Здесь определяется массив по имени `a2`, а функция `sizeof()` используется для гарантии равенства размеров массивов `a2` и `a1` (см. раздел 4.9, стр. 215). Затем происходит вызов функции `copy()` для копирования массива `a1` в массив `a2`. После вызова у элементов обоих массивов будут одинаковые значения.

Возвращенное функцией `copy()` значение является приращенным значением ее итератора назначения. Таким образом, итератор `ret` укажет на следующий элемент после последнего скопированного в массив `a2`.

Некоторые алгоритмы обладают так называемой версией “копирования”. Эти алгоритмы осуществляют некую обработку элементов исходной последовательности, но саму последовательность не изменяют. Они могут создавать новую последовательность, в которую и сохраняют результат обработки элементов исходной.

Например, алгоритм `replace()` читает последовательность и заменяет каждый экземпляр заданного значения другим значением. Алгоритм получает четыре параметра: два итератора, обозначающих исходный диапазон, и два значения. Он заменяет вторым значением значение каждого элемента, которое равно первому.

```
// заменить во всех элементах значение 0 на 42
replace(ilst.begin(), ilst.end(), 0, 42);
```

Этот вызов заменяет все экземпляры со значением 0 на 42. Если исходную последовательность следует оставить неизменной, необходимо применить алгоритм `replace_copy()`. Этой версии функции передают третий аргумент: итератор, указывающий получателя откорректированной последовательности.

```
// использовать функцию back_inserter() для увеличения контейнера
// назначения до необходимых размеров
replace_copy(ilst.cbegin(), ilst.cend(),
             back_inserter(ivec), 0, 42);
```

После этого вызова список `ilst` останется неизменным, а вектор `ivec` будет содержать копию его элементов, но со значением 42 вместо 0.



10.2.3. Алгоритмы, переупорядочивающие элементы контейнера

Некоторые алгоритмы изменяют порядок элементов в пределах контейнера. Яркий пример такого алгоритма — `sort()`. Вызов функции `sort()` упорядочивает элементы исходного диапазона в порядке сортировки, используя оператор `<` типа элемента.

Предположим, необходимо проанализировать слова, использованные в наборе детских рассказов. Текст рассказов содержится в векторе. Необходимо сократить этот вектор так, чтобы каждое слово присутствовало в нем только один раз, независимо от того, сколько раз оно встречается в любом из данных рассказов.

Для иллюстрации поставленной задачи используем в качестве исходного текста следующую простую историю:

```
the quick red fox jumps over the slow red turtle
```

В результате обработки этого текста программа должна создать следующий вектор:

fox	jumps	over	quick	red	red	slow	the	the	turtle
-----	-------	------	-------	-----	-----	------	-----	-----	--------

Упражнение 10.6. Напишите программу, использующую функцию `fill_n()` для обнуления последовательности целых чисел.

Упражнение 10.7. Определите, есть ли ошибки в следующих фрагментах кода, и, если есть, как их исправить:

```
(a) vector<int> vec; list<int> lst; int i;
    while (cin >> i)
        lst.push_back(i);
    copy(lst.cbegin(), lst.cend(), vec.begin());

(b) vector<int> vec;
    vec.reserve(10); // reserve рассматривается в разделе 9.4 (стр. 458)
    fill_n(vec.begin(), 10, 0);
```

Упражнение 10.8. Как упоминалось, алгоритмы не изменяют размер контейнеров, с которыми они работают. Почему использование функции `back_inserter()` не противоречит этому утверждению?

Устранение дубликатов

Для устранения повторяющихся слов сначала отсортируем вектор так, чтобы дубликаты располагались рядом друг с другом. После сортировки вектора можно использовать другой библиотечный алгоритм, `unique()`, чтобы расположить уникальные элементы в передней части вектора. Поскольку алгоритмы не могут работать с самими контейнерами, используем функцию-член `erase()` класса `vector` для фактического удаления элементов:

```
void elimDups(vector<string> &words)
{
    // сортировка слов в алфавитном порядке позволяет найти дубликаты
    sort(words.begin(), words.end());
    // функция unique() переупорядочивает исходный диапазон так, чтобы
    // каждое слово присутствовало только один раз в начальной части
    // диапазона, и возвращает итератор на элемент, следующий после
    // диапазона уникальных значений
    auto end_unique = unique(words.begin(), words.end());
    // для удаления не уникальных элементов используем
    // функцию erase() вектора
    words.erase(end_unique, words.end());
}
```

Алгоритм `sort()` получает два итератора, обозначающих диапазон элементов для сортировки. В данном случае сортируется весь вектор. После вызова функции `sort()` слова упорядочиваются так:

fox	jumps	over	quick	red	red	slow	the	the	turtle
-----	-------	------	-------	-----	-----	------	-----	-----	--------

Обратите внимание: слова `red` и `the` встречаются дважды.

Алгоритм `unique()`

После сортировки слов необходимо оставить только один экземпляр каждого из них. Алгоритм `unique()` перестраивает исходный диапазон так, чтобы устранить смежные повторяющиеся элементы, и возвращает итератор, обозначающий конец диапазона уникальных значений. После вызова функции `unique()` вектор выглядит так:

fox	jumps	over	quick	red	slow	the	turtle	???	???
-----	-------	------	-------	-----	------	-----	--------	-----	-----


`end_unique`
 (следующий элемент после последнего уникального)

Размер вектора `words` не изменился: в нем все еще десять элементов. Изменился только порядок этих элементов — смежные дубликаты были как бы “удалены”. Слово *удалены* заключено в кавычки потому, что функция `unique()` не удаляет элементы. Она переупорядочивает смежные дубликаты так, чтобы уникальные элементы располагались в начале последовательности. Возвращенный функцией `unique()` итератор указывает на следующий элемент после последнего уникального. Последующие элементы все еще существуют, но их значение уже не важно.



Библиотечные алгоритмы работают с итераторами, а не с контейнерами. Поэтому алгоритм не может непосредственно добавить или удалить элементы.

Применение функций контейнера для удаления элементов

Для фактического удаления неиспользуемых элементов следует использовать контейнерную функцию `erase()` (см. раздел 9.3.3, стр. 449). Удалению подлежит диапазон элементов от того, на который указывает итератор `end_unique`, и до конца контейнера `words`. После вызова контейнер `words` содержит восемь уникальных слов из исходного текста.

Следует заметить, что вызов функции `erase()` окажется безопасным, даже если вектор не содержит совпадающих слов. В этом случае функция `unique()` возвратит итератор, совпадающий с возвращенным функцией `words.end()`. Таким образом, оба аргумента функции `erase()` будут иметь одинаковое значение, а следовательно, обрабатываемый ею диапазон окажется пустым. Удаление пустого диапазона не приводит ни к какому результату, поэтому программа будет работать правильно даже тогда, когда в исходном тексте нет повторяющихся слов.

Упражнения раздела 10.2.3

Упражнение 10.9. Реализуйте собственную версию функции `elimDups()`. Проверьте ее в программе, выводящей содержимое вектора после чтения ввода, после вызова функции `unique()` и после вызова функции `erase()`.

Упражнение 10.10. Почему алгоритмы не изменяют размер контейнеров?

10.3. Перенастройка функций

Большинство алгоритмов сравнивает элементы исходной последовательности. По умолчанию такие алгоритмы используют оператор `<` или `==` типа элемента. Библиотека предоставляет также версии этих алгоритмов, позволяющие использовать собственный оператор вместо заданного по умолчанию.

Например, алгоритм `sort()` использует оператор `<` типа элемента. Но может понадобиться сортировать последовательность в порядке, отличном от определенного оператором `<`, либо у типа элемента последовательности может не быть оператора `<` (как у класса `Sales_data`). В обоих случаях необходимо переопределить стандартное поведение функции `sort()`.



10.3.1. Передача функций алгоритму

Предположим, например, что необходимо вывести вектор после вызова функции `elimDups()` (см. раздел 10.2.3, стр. 492). Однако слова должны быть упорядочены сначала по размеру, а затем в алфавитном порядке в пределах каждого размера. Чтобы переупорядочить вектор по длине слов, используем вторую перегруженную версию функции `sort()`. Она получает третий аргумент, называемый предикатом.

Предикаты

Предикат (predicate) — это допускающее вызов выражение, возвращающее значение, применимое в условии. Библиотечные алгоритмы используют *унарные предикаты* (unary predicate) (с одним параметром) или *бинарные предикаты* (binary predicate) (с двумя параметрами). Получающие предикаты алгоритмы вызывают его для каждого элемента в исходном диапазоне. Поэтому тип элемента должен допускать преобразование в тип параметра предиката.

Версия функции `sort()`, получающей бинарный предикат, использует его вместо оператора `<` при сравнении элементов. Предикаты, предоставляемые функции `sort()`, должны соответствовать требованиям, описанным в разделе 11.2.2 (стр. 543), а пока достаточно знать, что он должен определить единообразный порядок для всех возможных элементов в исходной последовательности. Функция `isShorter()` из раздела 6.2.2 (стр. 279) — хороший пример

функции, соответствующей этим требованиям. Таким образом, функцию `isShorter()` можно передать как предикат алгоритму `sort()`. Это переупорядочит элементы по размеру:

```
// функция сравнения, используемая при сортировке слов по длине
bool isShorter(const string &s1, const string &s2)
{
    return s1.size() < s2.size();
}
// сортировка слов по длине от коротких к длинным
sort(words.begin(), words.end(), isShorter);
```

Если контейнер `words` содержит те же данные, что и в разделе 10.2.3 (стр. 492), то этот вызов переупорядочит его так, что все слова длиной 3 символа расположатся перед словами длиной 4 символа, которые в свою очередь расположатся перед словами длиной 5 символов, и т.д.

Алгоритмы сортировки

При сортировке вектора `words` по размеру следует также обеспечить алфавитный порядок элементов одинаковой длины. Для обеспечения алфавитного порядка можно использовать алгоритм `stable_sort()`, обеспечивающий первоначальный порядок сортировки среди равных элементов.

Обычно об относительном порядке равных элементов можно не заботиться. В конце концов, они ведь равны. Но в данном случае под “равными” подразумеваются элементы со значениями одинаковой длины. Элементы одинаковой длины все еще отличаются друг от друга при просмотре их содержимого. Вызов функции `stable_sort()` позволяет расположить элементы с одинаковыми значениями длины в алфавитном порядке:

```
elimDups(words); // расположить слова в алфавитном порядке
// и удалить дубликаты
// пересортировать по длине, поддерживая алфавитный порядок среди слов
// той же длины
stable_sort(words.begin(), words.end(), isShorter);
for (const auto &s : words) // копировать строки не нужно
    cout << s << " "; // вывести каждый элемент, отделяя его пробелом
cout << endl;
```

Предположим, что перед этим вызовом вектор `words` был отсортирован в алфавитном порядке. После вызова он будет отсортирован по размеру элемента, а слова одинаковой длины остаются в алфавитном порядке. Если выполнить этот код для первоначального содержимого вектора, то результат будет таким:

```
fox red the over slow jumps quick turtle
```

Упражнения раздела 10.3.1

Упражнение 10.11. Напишите программу, использующую функции `stable_sort()` и `isShorter()` для сортировки вектора, переданного вашей версии функции `elimDups()`. Для проверки правильности программы выведите содержимое вектора.

Упражнение 10.12. Напишите функцию `compareIsbn()`, которая сравнивает члены `isbn` двух объектов класса `Sales_data`. Используйте эту функцию для сортировки вектора объектов класса `Sales_data`.

Упражнение 10.13. Библиотека определяет алгоритм `partition()`, получающий предикат и делящий контейнер так, чтобы значения, для которых предикат возвращает значение `true`, располагались в начале последовательности, а для которых он возвращает значение `false` — в конце. Алгоритм возвращает итератор на следующий элемент после последнего, для которого предикат возвратил значение `true`. Напишите функцию, которая получает строку и возвращает логическое значение, указывающее, содержит ли строка пять символов или больше. Используйте эту функцию для разделения вектора `words`. Выведите элементы, у которых есть пять или более символов.

10.3.2. Лямбда-выражения

У передаваемых алгоритмам предикатов должен быть точно один или два параметра, в зависимости от того, использует ли алгоритм унарный или бинарный предикат соответственно. Но иногда необходима обработка, которая требует большего количества аргументов, чем позволяет предикат алгоритма. Например, решение для последнего упражнения в предыдущем разделе имело жестко заданный размер в 5 символов, согласно которому предикат делил последовательность. Было бы удобней иметь возможность разделять последовательность без необходимости писать отдельный предикат для каждого возможного размера.

Для примера пересмотрим программу из раздела 10.3.1 (стр. 496) так, чтобы вывести количество слов с указанным размером или больше него. Вывод изменим так, чтобы он сообщал только те слова, длина которых равна или больше заданной.

Вот “эскиз” этой функции, которую мы назовем `biggies()`:

```
void biggies(vector<string> &words,
            vector<string>::size_type sz)
{
    elimDups(words); // расположить слова в алфавитном порядке
    // и удалить дубликаты
    // пересортировать по длине, поддерживая алфавитный порядок среди слов
    // той же длины
    stable_sort(words.begin(), words.end(), isShorter);
    // получить итератор на первый элемент, размер которого >= sz
```

```
// вычислить количество элементов с размером >= sz
// вывести слова, размер которых равен или больше заданного, разделяя
// их пробелами
}
```

Новая проблема — поиск первого элемента вектора заданного размера. Как известно, чтобы выяснить количество элементов, размер которых равен или больше заданного, можно использовать их позицию.

Для поиска элементов определенного размера можно использовать библиотечный алгоритм `find_if()`. Подобно функции `find()` (см. раздел 10.1, стр. 482), алгоритм `find_if()` получает два итератора, обозначающих диапазон. В отличие от функции `find()`, третий аргумент функции `find_if()` является предикатом. Алгоритм `find_if()` вызывает переданный предикат для каждого элемента в исходном диапазоне. Он возвращает первый элемент, для которого предикат возвращает отличное от нуля значение, или конечный итератор, если ни один подходящий элемент не найден.

Совсем не сложно написать функцию, которая получает строку и размер, а возвращает логическое значение, означающее, не превосходит ли размер данной строки указанный. Однако функция `find_if()` получает унарный предикат, поэтому любая передаваемая ей функция, которая может быть вызвана с элементом исходной последовательности, должна иметь только один параметр. Нет никакого способа передать ей второй аргумент, представляющий размер. Чтобы решить эту часть проблемы, используем некоторые дополнительные средства языка.

Знакомство с лямбда-выражением

Алгоритму можно передать любой вид *вызываемого объекта* (callable object). Объект или выражение является вызываемым, если к нему можно применить оператор вызова (см. раздел 1.5.2, стр. 51). Таким образом, если *e* — вызываемое выражение, то можно написать *e* (*аргументы*), где *аргументы* — это разделяемый запятыми список любого количества аргументов.

Единственными вызываемыми объектами, использованными до сих пор, были функции и указатели на функции (см. раздел 6.7, стр. 322). Есть еще два вида вызываемых объектов: классы, перегружающие оператор вызова функции (будут рассматриваться в разделе 14.8 на стр. 722), и *лямбда-выражения* (lambda expression).

**C++
11**

Лямбда-выражение представляет собой вызываемый блок кода. Его можно считать безымянной встраиваемой функцией. Подобно любой функции, у лямбда-выражений есть тип возвращаемого значения, список параметров и тело функции. В отличие от функции, лямбда-выражения могут быть определены в функции. Форма лямбда-выражений такова:

```
[список захвата] (список параметров) -> тип возвращаемого значения {тело функции}
```

где *список захвата* (зачастую пустой) — это список локальных переменных, определенных в содержащей функции; *тип возвращаемого значения*, *список параметров* и *тело функции* — те же самые, что и у любой обычной функции. Однако, в отличие от обычных функций, для определения типа возвращаемого значения лямбда-выражения должны использовать замыкающий тип (см. раздел 6.3.3, стр. 301).

Список параметров и типа возвращаемого значения могут отсутствовать, но список захвата и тело функции должны быть всегда:

```
auto f = [] { return 42; };
```

Здесь *f* определено как вызываемый объект, не получающий никаких аргументов и возвращающий значение 42. Вызов лямбда-выражений происходит таким же способом, что и вызов функций, — при помощи оператора вызова:

```
cout << f() << endl; // выводит 42
```

Пропуск круглых скобок и списка параметров в лямбда-выражении эквивалентен определению пустого списка параметров. Следовательно, когда происходит вызов лямбда-выражения *f*, список аргументов оказывается пустым. Если пропущен тип возвращаемого значения, то выведенный тип возвращаемого значения лямбда-выражения будет зависеть от кода в теле функции. Если телом является только оператор *return*, тип возвращаемого значения выводится из типа возвращаемого выражения. В противном случае типом возвращаемого значения является *void*.



Лямбда-выражения, тела которых содержат нечто кроме одного оператора *return*, что не определяет тип возвращаемого значения, возвращают тип *void*.

Передача аргументов лямбда-выражению

Подобно вызовам обычных функций, аргументы вызова лямбда-выражения используются для инициализации его параметров. Как обычно, типы аргумента и параметра должны совпадать. В отличие от обычных функций, у лямбда-выражений не может быть аргументов по умолчанию (см. раздел 6.5.1, стр. 308). Поэтому у вызова лямбда-выражения всегда столько аргументов, сколько и параметров. Как только параметры инициализируются, выполняется тело лямбда-выражения.

Для примера передачи аргументов можно написать лямбда-выражение, ведущее себя как функция *isShorter()*:

```
[](const string &a, const string &b)
{ return a.size() < b.size(); }
```

Пустой список захвата означает, что это лямбда-выражение не будет использовать локальных переменных из окружающей функции. Параметры лямбда-выражения, как и параметры функции `isShorter()`, будут ссылкой на константную строку. Как и тело функции `isShorter()`, тело лямбда-выражения сравнивает размер параметров и возвращает логическое значение, зависящее от соотношения размеров своих аргументов.

Вызов функции `stable_sort()` можно переписать так, чтобы использовать это лямбда-выражение следующим образом:

```
// сортировать слова по размеру, поддерживая алфавитный порядок среди
// слов того же размера
stable_sort(words.begin(), words.end(),
            [](const string &a, const string &b)
            { return a.size() < b.size(); });
```

Когда функция `stable_sort()` будет сравнивать два элемента, она вызовет данное лямбда-выражение.

Использование списка захвата

Теперь все готово для решения первоначальной задачи — создания вызываемого выражения, которое можно передать функции `find_if()`. Необходимо выражение, сравнивающее длину каждой строки в исходной последовательности со значением параметра `sz` функции `biggies()`.

Хотя лямбда-выражение может присутствовать в функции, оно способно использовать локальные переменные этой функции, *только* заранее определив, какие из них предстоит использовать. Лямбда-выражение определяет подлежащие использованию локальные переменные, включив их в *список захвата* (capture list). Список захвата предписывает лямбда-выражению включить информацию, необходимую для доступа к этим переменным, в само лямбда-выражение.

В данном случае лямбда-выражение захватит переменную `sz` и будет иметь один параметр типа `string`. Тело лямбда-выражения сравнивает размер переданной строки с захваченным значением переменной `sz`:

```
[sz](const string &a)
{ return a.size() >= sz; };
```

В начинающих лямбда-выражение квадратных скобках, `[]`, можно расположить разделяемый запятыми список имен, определенных в окружающей функции.

Поскольку данное лямбда-выражение захватывает переменную `sz`, ее можно будет использовать в теле лямбда-выражения. Лямбда-выражение не захватывает вектор `words`, поэтому доступа к его переменным она не имеет. Если бы лямбда-выражение имело пустой список захвата, наш код не компилировался бы:

```
// ошибка: sz не захвачена
[](const string &a)
{ return a.size() >= sz; };
```



Лямбда-выражение может использовать локальную переменную окружающей функции, *только если* она присутствует в ее списке захвата.

Вызов функции `find_if()`

Используя это лямбда-выражение, можно найти первый элемент, размер которого не меньше `sz`:

```
// получить итератор на первый элемент, размер которого >= sz
auto wc = find_if(words.begin(), words.end(),
    [sz](const string &a)
    { return a.size() >= sz; });
```

Вызов функции `find_if()` возвращает итератор на первый элемент, длина которого не меньше `sz`, или на элемент `words.end()`, если такового элемента не существует.

Возвращенный функцией `find_if()` итератор можно использовать для вычисления количества элементов, расположенных между этим итератором и концом вектора `words` (см. раздел 3.4.2, стр. 159):

```
// вычислить количество элементов с размером >= sz
auto count = words.end() - wc;
cout << count << " " << make_plural(count, "word", "s")
    << " of length " << sz << " or longer" << endl;
```

Для вывода в сообщении слова `word` или `words`, в зависимости от того, равен ли размер 1, оператор вывода вызывает функцию `make_plural()` (см. раздел 6.3.2, стр. 294).

Алгоритм `for_each()`

Последняя часть задачи — вывод тех элементов вектора `words`, длина которых не меньше `sz`. Для этого используем алгоритм `for_each()`, получающий вызываемый объект и вызывающий его для каждого элемента в исходном диапазоне:

```
// вывести слова, размер которых равен или больше заданного, разделяя
// их пробелами
for_each(wc, words.end(),
    [](const string &s){cout << s << " ";});
cout << endl;
```

Список захвата этого лямбда-выражения пуст, но все же его тело использует два имени: его собственный параметр `s` и `cout`.

Список захвата пуст, поскольку он используется только для нестатических переменных, определенных в окружающей функции. Лямбда-выражение вполне может использовать имена, определенные вне той функции, в которой присутствует лямбда-выражение. В данном случае имя `cout` не локально определено в функции `biggies()`, оно определено в заголовке `iostream`. Пока заголовок `iostream` находится в области видимости функции `biggies()`, данное лямбда-выражение может использовать имя `cout`.



Список захвата используется только для локальных нестатических переменных; лямбда-выражения могут непосредственно использовать статические локальные переменные и переменные, объявленные вне функции.

Объединив все вместе

Теперь, изучив элементы программы подробно, рассмотрим ее в целом:

```
void biggies(vector<string> &words,
            vector<string>::size_type sz)
{
    elimDups(words); // расположить слова в алфавитном порядке
    // и удалить дубликаты
    // пересортировать по длине, поддерживая алфавитный порядок среди слов
    // той же длины
    stable_sort(words.begin(), words.end(),
                [](const string &a, const string &b)
                { return a.size() < b.size(); });
    // получить итератор на первый элемент, размер которого >= sz
    auto wc = find_if(words.begin(), words.end(),
                     [sz](const string &a)
                     { return a.size() >= sz; });
    // вычислить количество элементов с размером >= sz
    auto count = words.end() - wc;
    cout << count << " " << make_plural(count, "word", "s")
          << " of length " << sz << " or longer" << endl;
    // вывести слова, размер которых равен или больше заданного, разделяя
    // их пробелами
    for_each(wc, words.end(),
             [](const string &s){ cout << s << " "; });
    cout << endl;
}
```

Упражнения раздела 10.3.2

Упражнение 10.14. Напишите лямбда-выражение, получающее два целых числа и возвращающее их сумму.

Упражнение 10.15. Напишите лямбда-выражение, захватывающее переменную типа `int` окружающей функции и получающей параметр типа

`int`. Лямбда-выражение должно вернуть сумму захваченного значения типа `int` и параметра типа `int`.

Упражнение 10.16. Напишите собственную версию функции `biggies()`, используя лямбда-выражения.

Упражнение 10.17. Перепишите упражнение 10.12 из раздела 10.3.1 (стр. 496) так, чтобы в вызове функции `sort()` вместо функции `compareIsbn()` использовалось лямбда-выражение.

Упражнение 10.18. Перепишите функцию `biggies()` так, чтобы использовать алгоритм `partition()` вместо алгоритма `find_if()`. Алгоритм `partition()` описан в упражнении 10.13 раздела 10.3.1 (стр. 496).

Упражнение 10.19. Перепишите предыдущее упражнение так, чтобы использовать алгоритм `stable_partition()`, который, подобно алгоритму `stable_sort()`, обеспечивает исходный порядок элементов в разделяемой последовательности.

10.3.3. Захват и возвращение значений лямбда-выражениями

При определении лямбда-выражения компилятор создает новый (безымянный) класс, соответствующий этому лямбда-выражению. Создание этих классов рассматривается в разделе 14.8.1 (стр. 724), а пока следует понять, что при передаче лямбда-выражения функции определяется новый тип и создается его объект. Безымянный объект этого созданного компилятором типа и передается как аргумент. Аналогично при использовании ключевого слова `auto` для определения переменной, инициализированной лямбда-выражением, определяется объект типа, созданного из этого лямбда-выражения.

По умолчанию созданный из лямбда-выражения класс содержит переменные-члены, соответствующие захваченным переменным лямбда-выражения. Подобно переменным-членам любого класса, переменные-члены лямбда-выражения инициализируются при создании его объекта.

Захват по значению

Подобно передаче параметров, переменные можно захватывать по значению или по ссылке. В табл. 10.1 (стр. 503) приведены различные способы создания списка захвата. До сих пор у использованных лямбда-выражений захват переменных осуществлялся по значению. Подобно передаче по значению параметров, захват переменной по значению подразумевает ее копирование. Но, в отличие от параметров, копирование значения при захвате осуществляется при создании лямбда-выражения, а не при его вызове:

```
void fcn1()
{
    size_t v1 = 42; // локальная переменная
    // копирует v1 в вызываемый объект f
    auto f = [v1] { return v1; };
    v1 = 0;
    auto j = f(); // j = 42; f получит копию v1 на момент создания
}
```

Поскольку значение копируется при создании лямбда-выражения, последующие изменения захваченной переменной никак не влияют на соответствующее значение в лямбда-выражении.

Таблица 10.1. Список захвата лямбда-выражения

<code>[]</code>	Пустой список захвата. Лямбда-выражение не может использовать переменные из содержащей функции. Лямбда-выражение может использовать локальные переменные, только если оно захватывает их
<code>[names]</code>	<i>names</i> — разделяемый запятыми список имен, локальных для содержащей функции. По умолчанию переменные в списке захвата копируются. Имя, которому предшествует знак <code>&</code> , захватывается по ссылке
<code>[&]</code>	Неявный захват по ссылке. Сущности из содержащей функции используются в теле лямбда-выражения по ссылке
<code>[=]</code>	Неявный захват по значению. Сущности из содержащей функции используются в теле лямбда-выражения как копии
<code>[&, identifier_list]</code>	<i>identifier_list</i> — разделяемый запятыми список любого количества переменных из содержащей функции. Эти переменные захватываются по значению; любые неявно захваченные переменные захватываются по ссылке. Именам в списке <i>identifier_list</i> не могут предшествовать символы <code>&</code>
<code>[=, reference_list]</code>	Переменные, включенные в список <i>reference_list</i> , захватываются по ссылке; любые неявно захваченные переменные захватываются по значению. Имена в списке <i>reference_list</i> не могут включать часть <code>this</code> и должны предваряться символом <code>&</code>

Захват по ссылке

Можно также определять лямбда-выражения, захватывающие переменные по ссылке. Например:

```
void fcn2()
{
    size_t v1 = 42; // локальная переменная
    // объект f2 содержит ссылку на v1
```

```

auto f2 = [&v1] { return v1; };
v1 = 0;
auto j = f2(); // j = 0; f2 ссылается на v1; он не хранится в j
}

```

Символ `&` перед `v1` указывает, что переменная `v1` должна быть захвачена как ссылка. Захваченная по ссылке переменная действует так же, как любая другая ссылка. При использовании переменной в теле лямбда-выражения фактически применяется объект, с которым связана эта ссылка. В данном случае, когда лямбда-выражение возвращает `v1`, возвращается значение объекта, на который ссылается переменная `v1`.

Захват ссылок имеет те же проблемы и ограничения, что и возвращение ссылок (см. раздел 6.1.1, стр. 295). При захвате переменной по ссылке следует быть *уверенным*, что объект, на который она ссылается, существует на момент выполнения лямбда-выражения. Переменные, захваченные лямбда-выражением, являются локальными, они перестают существовать сразу, как только функция завершится. Если лямбда-выражение продолжит выполняться после завершения функции, то используемые ею локальные переменные окажутся несуществующими.

Иногда захват по ссылке необходим. Например, может понадобиться, чтобы функция `biggies()` получала ссылку на поток `ostream` для записи символа, используемого как разделитель:

```

void biggies(vector<string> &words,
             vector<string>::size_type sz,
             ostream &os = cout, char c = ' ')
{
    // код, переупорядочивающий слова как прежде
    // оператор вывода количества, пересмотренный для вывода os
    for_each(words.begin(), words.end(),
             [&os, c](const string &s) { os << s << c; });
}

```

Объекты потока `ostream` нельзя копировать (см. раздел 8.1.1, стр. 401); единственный способ захвата объекта `os` — это ссылка (или указатель).

При передаче лямбда-выражения функции, как и в случае вызова функции `for_each()`, лямбда-выражение выполняется немедленно. Захват объекта `os` по ссылке хорош потому, что переменные в функции `biggies()` существуют во время выполнения функции `for_each()`.

Лямбда-выражение можно также вернуть из функции. Функция может вернуть вызываемый объект непосредственно или вернуть объект класса, у которого вызываемый объект является переменной-членом. Если функция возвращает лямбда-выражение, то по тем же причинам, по которым функция не должна возвращать ссылку на локальную переменную, лямбда-выражение не должно содержать захваченных ссылок.



Когда переменная захвачена по ссылке, следует удостовериться, что эта переменная существует во время выполнения лямбда-выражения.

СОВЕТ. НЕ УСЛОЖНЯЙТЕ СПИСКИ ЗАХВАТА ЛЯМБДА-ВЫРАЖЕНИЙ

Механизм захвата лямбда-выражения хранит полученную информацию между моментом создания лямбда-выражения (т.е. когда выполняется код определения лямбда-выражения) и моментом собственно выполнения лямбда-выражения. Разработчику следует самостоятельно позаботиться о том, чтобы независимо от момента захвата информации она осталась достоверной на момент выполнения лямбда-выражения.

Захват обычной переменной (типа `int`, `string` и так далее, но не указателя) обычно достаточно прост. В данном случае следует позаботиться о наличии у переменной значения в момент ее захвата.

При захвате указателя, итератора или переменной по ссылке следует удостовериться, что связанный с ними объект все еще существует на момент выполнения лямбда-выражения. Кроме того, объект в этот момент гарантированно должен иметь значение. Код, выполняемый между моментом создания лямбда-выражения и моментом его выполнения, может изменить значение объекта, на который указывает (или ссылается) захваченная сущность. Во время захвата указателя (или ссылки) значение объекта, возможно, и было правильным, но ко времени выполнения лямбда-выражения оно могло измениться.

Как правило, сокращая объем захватываемых данных, потенциальных проблем с захватом можно избежать. Кроме того, по возможности избегайте захвата указателей и ссылок.

Неявный захват

Вместо предоставления явного списка переменных содержащей функции, которые предстоит использовать, можно позволить компилятору самостоятельно вывести используемые переменные из кода тела лямбда-выражения. Чтобы заставить компилятор самостоятельно вывести список захвата, в нем используется символ `&` или `=`. Символ `&` указывает, что предполагается захват по ссылке, а символ `=` — что значения захватываются по значению. Например, передаваемое функции `find_if()` лямбда-выражение можно переписать так:

```
// sz неявно захватывается по значению
wc = find_if(words.begin(), words.end(),
            [=](const string &s)
            { return s.size() >= sz; });
```

Если одни переменные необходимо захватить по значению, а другие по ссылке, вполне можно совместить явный и неявный захваты:

```
void biggies(vector<string> &words,
            vector<string>::size_type sz,
            ostream &os = cout, char c = ' ')
{
    // другие действия, как прежде
    // os неявно захватывается по ссылке;
    // s явно захватывается по значению
    for_each(words.begin(), words.end(),
             [&, c](const string &s) { os << s << c; });
    // os явно захватывается по ссылке;
    // s неявно захватывается по значению
    for_each(words.begin(), words.end(),
             [=, &os](const string &s) { os << s << c; });
}
```

При совмещении неявного и явного захвата первым элементом в списке захвата должен быть символ & или =. Эти символы задают режим захвата по умолчанию: по ссылке или по значению соответственно.

При совмещении неявного и явного захвата явно захваченные переменные должны использовать дополнительную форму. Таким образом, при неявном захвате по ссылке (с использованием &) явно именованные переменные должны захватываться по значению; следовательно, их именам не может предшествовать символ &. И наоборот, при неявном захвате по значению (с использованием =) явно именованным переменным должен предшествовать символ &, означающий, что они должны быть захвачены по ссылке.

Изменяемые лямбда-выражения

По умолчанию лямбда-выражение не может изменить значение переменной, которую она копирует по значению. Чтобы изменить значение захваченной переменной, за списком параметров должно следовать ключевое слово `mutable`. Изменяемые лямбда-выражения не могут пропускать список параметров:

```
void fcn3()
{
    size_t v1 = 42; // локальная переменная
    // f может изменить значение захваченных переменных
    auto f = [v1] () mutable { return ++v1; };
    v1 = 0;
    auto j = f(); // j = 43
}
```

Может ли захваченная по ссылке переменная быть изменена, зависит только от того, ссылается ли она на константный или неконстантный тип:

```
void fcn4()
{
```

```

size_t v1 = 42; // локальная переменная
// v1 - ссылка на неконстантную переменную
// эту переменную можно изменить в f2 при помощи ссылки
auto f2 = [&v1] { return ++v1; };
v1 = 0;
auto j = f2(); // j = 1
}

```

Определение типа возвращаемого значения лямбда-выражения

Использованные до сих пор лямбда-выражения содержали только один оператор `return`. В результате тип возвращаемого значения определять было не нужно. По умолчанию, если тело лямбда-выражения содержало какие-нибудь операторы, кроме оператора `return`, то подразумевалось, что оно возвращало тип `void`. Подобно другим функциям, возвращающим тип `void`, подобные лямбда-выражения могут не возвращать значения.

В качестве примера используем библиотечный алгоритм `transform()` и лямбда-выражение для замены каждого отрицательного значения в последовательности его абсолютным значением:

```

transform(vi.begin(), vi.end(), vi.begin(),
[] (int i) { return i < 0 ? -i : i; });

```

Функция `transform()` получает три итератора и вызываемый объект. Первые два итератора обозначают исходную последовательность, третий итератор — назначение. Алгоритм вызывает переданный ему вызываемый объект для каждого элемента исходной последовательности и записывает результат по назначению. Как и в данном примере, итератор назначения может быть тем же, обозначающим начало ввода. Когда исходный итератор и итератор назначения совпадают, алгоритм `transform()` заменяет каждый элемент в исходном диапазоне результатом вызова вызываемого объекта для этого элемента.

В этом вызове передавалось лямбда-выражение, которое возвращает абсолютное значение своего параметра. Тело лямбда-выражения — один оператор `return`, который возвращает результат условного выражения. Необходимости определять тип возвращаемого значения нет, поскольку его можно вывести из типа условного оператора.

Но если написать на первый взгляд эквивалентную программу, используя оператор `if`, то код не будет компилироваться:

```

// ошибка: нельзя вывести тип возвращаемого значения лямбда-выражения
transform(vi.begin(), vi.end(), vi.begin(),
[] (int i) { if (i < 0) return -i; else return i; });

```

Эта версия лямбда-выражения выводит тип возвращаемого значения как `void`, но возвращает значение.



Когда необходимо определить тип возвращаемого значения для лямбда-выражения, следует использовать замыкающий тип возвращаемого значения (см. раздел 6.3.3, стр. 301):

```
transform(vi.begin(), vi.end(), vi.begin(),
        [](int i) -> int
        { if (i < 0) return -i; else return i; });
```

В данном случае четвертым аргументом функции `transform()` является лямбда-выражение с пустым списком захвата, единственным параметром типа `int` и возвращаемым значением типа `int`. Его телом является оператор `if`, возвращающий абсолютное значение параметра.

Упражнения раздела 10.3.3

Упражнение 10.20. Библиотека определяет алгоритм `count_if()`. Подобно алгоритму `find_if()`, он получает пару итераторов, обозначающих исходный диапазон и предикат, применяемый к каждому элементу заданного диапазона. Функция `count_if()` возвращает количество раз, когда предикат вернул значение `true`. Используйте алгоритм `count_if()`, чтобы переписать ту часть программы, которая рассчитывала количество слов длиной больше 6.

Упражнение 10.21. Напишите лямбда-выражение, которое захватывает локальную переменную типа `int` и осуществляет декремент ее значения, пока оно не достигает 0. Как только значение переменной достигнет 0, декремент переменной прекращается. Лямбда-выражение должно вернуть логическое значение, указывающее, имеет ли захваченная переменная значение 0.



10.3.4. Привязка аргументов

Лямбда-выражения особенно полезны для простых операций, которые не предполагается использовать в более чем одном или двух местах. Если ту же операцию необходимо осуществлять во многих местах, то обычно определяют функцию, а не повторяют то же лямбда-выражение многократно. Аналогично, если операция требует многих операторов, обычно лучше использовать функцию.

Как правило, вместо лямбда-выражения с пустым списком захвата проще использовать функцию. Как уже упоминалось, для упорядочивания вектора по длине слов можно использовать или лямбда-выражение, или нашу функцию `isShorter()`. Точно так же совсем не сложно заменить лямбда-выражение, выводившее содержимое вектора, функцией, которая получает строку и выводит ее на стандартное устройство вывода.

Однако не так просто написать функцию для замены лямбда-выражений, которые захватывают локальные переменные. Рассмотрим, например, использованное в вызове функции `find_if()` лямбда-выражение, которое сравнивало размер строки с заданным размером. Совсем не сложно написать функцию, выполняющую те же действия:

```
bool check_size(const string &s, string::size_type sz)
{
    return s.size() >= sz;
}
```

Но мы не можем использовать эту функцию как аргумент функции `find_if()`. Как уже упоминалось, функция `find_if()` получает унарный предикат, поэтому переданное ей вызываемое выражение должно получать один аргумент. Лямбда-выражение, переданное функцией `biggies()` функции `find_if()`, использует свой список захвата для хранения значения переменной `sz`. Чтобы использовать функцию `check_size()` вместо этого лямбда-выражения, следует выяснить, как передать аргумент `sz` параметру.

Библиотечная функция `bind()`

C++ 11 Проблему передачи аргумента размера функции `check_size()` можно решить при помощи новой библиотечной функции `bind()`, определенной в заголовке `functional`. Функцию `bind()` можно считать универсальным адаптером функции (см. раздел 9.6, стр. 473). Она получает вызываемый объект и создает новый вызываемый объект, который адаптирует список параметров исходного объекта.

Общая форма вызова функции `bind()` такова:

```
auto новыйВызываемыйОбъект = bind(вызываемыйОбъект, список_аргументов);
```

где `новыйВызываемыйОбъект` — это новый вызываемый объект, а `список_аргументов` — разделяемый запятыми список аргументов, соответствующих параметрам переданного вызываемого объекта `вызываемыйОбъект`. Таким образом, когда происходит вызов объекта `новыйВызываемыйОбъект`, он вызывает `вызываемыйОбъект`, передавая аргументы из списка `список_аргументов`.

Аргументы из списка `список_аргументов` могут включать имена в формате `_n`, где `n` — целое число. Эти аргументы — *знакоместа*, представляющие параметры объекта `новыйВызываемыйОбъект`. Они располагаются *вместо* аргументов, которые будут переданы объекту `новыйВызываемыйОбъект`. Число `n` является позицией параметра вновь созданного вызываемого объекта: `_1` — первый параметр, `_2` — второй и т.д.

Привязка параметра `sz` к функции `check_size()`

В качестве примера использования функции `bind()` создадим объект, который вызывает функцию `check_size()` с фиксированным значением ее параметра размера:

```
// check6 - вызываемый объект, получающий один аргумент типа string
// и вызывающий функцию check_size() с этой строкой и значением 6
auto check6 = bind(check_size, _1, 6);
```

У этого вызова функции `bind()` есть только одно знакоместо, означающее, что вызываемый объект `check6()` получает один аргумент. Знакоместо располагается первым в списке аргументов. Это означает, что параметр вызываемого объекта `check6()` соответствует первому параметру функции `check_size()`. Этот параметр имеет тип `const string&`, а значит, параметр вызываемого объекта `check6()` также имеет тип `const string&`. Таким образом, при вызове `check6()` следует передать аргумент типа `string`, который вызываемый объект `check6()` передаст в качестве первого аргумента функции `check_size()`.

Второй аргумент в списке аргументов (т.е. третий аргумент функции `bind()`) является значением 6. Это значение связывается со вторым параметром функции `check_size()`. Каждый раз, когда происходит вызов вызываемого объекта `check6()`, он передает функции `check_size()` значение 6 как второй аргумент:

```
string s = "hello";
bool b1 = check6(s); // check6(s) вызывает check_size(s, 6)
```

Используя функцию `bind()`, можно заменить следующий исходный вызов функции `find_if()` на базе лямбда-выражения:

```
auto wc = find_if(words.begin(), words.end(),
                 [sz](const string &a)
```

Кодом, использующим функцию `check_size()`, заменить

```
auto wc = find_if(words.begin(), words.end(),
                 bind(check_size, _1, sz));
```

Этот вызов функции `bind()` создает вызываемый объект, который привязывает второй аргумент функции `check_size()` к значению параметра `sz`. Когда функция `find_if()` вызовет этот объект для строк вектора `words`, он, в свою очередь, вызовет функцию `check_size()`, передав полученную строку и значение параметра `sz`. Таким образом, функция `find_if()` фактически вызовет функцию `check_size()` для каждой строки в исходном диапазоне и сравнит размер этой строки со значением параметра `sz`.

Использование имен из пространства имен `placeholders`

Имена формата `_n` определяются в пространстве имен `placeholders`. Само это пространство имен определяется в пространстве имен `std` (см. раздел 3.1, стр. 124). Чтобы использовать эти имена, следует предоставить имена обоих пространств имен. Подобно нашим другим примерам, данные вызовы функции `bind()` подразумевали наличие соответствующих объявлений `using`. Рассмотрим объявление `using` для имени `_1`:

```
using std::placeholders::_1;
```

Это объявление свидетельствует о том, что используется имя `_1`, определенное в пространстве имен `placeholders`, которое само определено в пространстве имен `std`.

Для каждого используемого имени знакоместа следует предоставить отдельное объявление `using`. Но поскольку написание таких объявлений может быть утомительно и ведет к ошибкам, вместо этого можно использовать другую форму `using`, которая подробно рассматривается в разделе 18.2.2 (стр. 991):

```
using namespace пространствоимен_имя;
```

Она свидетельствует, что необходимо сделать доступными для нашей программы все имена из пространства имен `пространствоимен_имя`:

```
using namespace std::placeholders;
```

Этот код позволяет использовать все имена, определенные в пространстве имен `placeholders`. Подобно функции `bind()`, пространство имен `placeholders` определено в заголовке `functional`.

Аргументы функции `bind()`

Как уже упоминалось, функцию `bind()` можно использовать для фиксированного значения параметра. В более общем случае функцию `bind()` можно использовать для привязки или перестройки параметров в предоставленном вызываемом объекте. Предположим, например, что `f()` — вызываемый объект с пятью параметрами:

```
// g - вызываемый объект, получающий два аргумента
auto g = bind(f, a, b, _2, c, _1);
```

Этот вызов функции `bind()` создает новый вызываемый объект, получающий два аргумента, представленные знакоместами `_2` и `_1`. Новый вызываемый объект передает собственные аргументы как третий и пятый аргументы вызываемому объекту `f()`. Первый, второй и четвертый аргументы вызываемого объекта `f()` связаны с переданными значениями `a`, `b` и `c` соответственно.

Аргументы вызываемого объекта `g()` связаны со знакоместами по позиции. Таким образом, первый аргумент вызываемого объекта `g()` связан с парамет-

ром `_1`, а второй — с параметром `_2`. Следовательно, когда происходит вызов `g()`, его первый аргумент будет передан как последний аргумент вызываемого объекта `f()`; второй аргумент `g()` будет передан как третий. В действительности этот вызов функции `bind()` преобразует вызов `g(_1, _2)` в вызов `f(a, b, _2, c, _1)`.

Таким образом, вызов вызываемого объекта `g()` вызывает вызываемый объект `f()` с использованием аргументов вызываемого объекта `g()` для знакомств наряду с аргументами `a`, `b` и `c`. Например, вызов `g(X, Y)` приводит к вызову `f(a, b, Y, c, X)`.

Использование функции `bind()` для переупорядочивания параметров

Рассмотрим более конкретный пример применения функции `bind()` для переупорядочивания аргументов. Используем ее для обращения смысла функции `isShorter()` следующим образом:

```
// сортировка по длине слов от коротких к длинным
sort(words.begin(), words.end(), isShorter);
// сортировка по длине слов от длинных к коротким
sort(words.begin(), words.end(), bind(isShorter, _2, _1));
```

В первом вызове, когда алгоритм `sort()` должен сравнить два элемента, `A` и `B`, он вызовет функцию `isShorter(A, B)`. Во втором вызове аргументы функции `isShorter()` меняются местами. В данном случае, когда алгоритм `sort()` сравнивает элементы, он вызывает функцию `isShorter(B, A)`.

Привязка ссылочных параметров

По умолчанию аргументы функции `bind()`, не являющиеся знакоместами, копируются в возвращаемый ею вызываемый объект. Однако, подобно лямбда-выражениям, иногда необходимо связать аргументы, которые следует передать по ссылке, или необходимо связать аргумент, тип которого не допускает копирования.

Для примера заменим лямбда-выражение, которое захватывало поток `ostream` по ссылке:

```
// os - локальная переменная, ссылающаяся на поток вывода
// c - локальная переменная типа char
for_each(words.begin(), words.end(),
         [&os, c](const string &s) { os << s << c; });
```

Вполне можно написать функцию, выполняющую ту же задачу:

```
ostream &print(ostream &os, const string &s, char c)
{
    return os << s << c;
}
```

Но для замены захвата переменной `os` нельзя использовать функцию `bind()` непосредственно:

```
// ошибка: нельзя копировать os
for_each(words.begin(), words.end(), bind(print, os, _1, ' '));
```

Поскольку функция `bind()` копирует свои аргументы, она не сможет скопировать поток `ostream`. Если объект необходимо передать функции `bind()`, не копируя, то следует использовать библиотечную функцию `ref()`:

```
for_each(words.begin(), words.end(),
         bind(print, ref(os), _1, ' '));
```

Функция `ref()` возвращает объект, который содержит переданную ссылку, являясь при этом вполне копируемым. Существует также функция `cref()`, создающая класс, содержащий ссылку на константу. Подобно функции `bind()`, функции `ref()` и `cref()` определены в заголовке `functional`.

СОВМЕСТИМОСТЬ С ПРЕЖНЕЙ ВЕРСИЕЙ: ПРИВЯЗКА АРГУМЕНТОВ

Прежние версии языка C++ имели много больше ограничений, и все же более сложный набор средств привязки аргументов к функциям. Библиотека определяет две функции — `bind1st()` и `bind2nd()`. Подобно функции `bind()`, эти функции получают функцию и создают новый вызываемый объект для вызова переданной функции с одним из ее параметров, связанным с переданным значением. Но эти функции могут связать только первый или второй параметр соответственно. Поскольку они имеют очень много ограничений, в новом стандарте эти функции *не рекомендуются*. Это устаревшее средство, которое может не поддерживаться в будущих выпусках. Современные программы C++ должны использовать функцию `bind()`.

Упражнения раздела 10.3.4

Упражнение 10.22. Перепишите программу подсчета слов размером 6 символов с использованием функций вместо лямбда-выражений.

Упражнение 10.23. Сколько аргументов получает функции `bind()`?

Упражнение 10.24. Используйте функции `bind()` и `check_size()` для поиска первого элемента вектора целых чисел, значение которого больше длины заданного строкового значения.

Упражнение 10.25. В упражнениях раздела 10.3.2 (стр. 501) была написана версия функции `biggies()`, использующая алгоритм `partition()`. Перепишите эту функцию так, чтобы использовать функции `check_size()` и `bind()`.

10.4. Возвращаясь к итераторам

В дополнение к итераторам, определяемым для каждого из контейнеров, библиотека определяет в заголовке `iterator` несколько дополнительных видов итераторов.

- *Итератор вставки* (insert iterator). Связан с контейнером и применяется для вставки элементов в контейнер.
- *Потоковый итератор* (stream iterator). Может быть связан с потоком ввода или вывода и применяется для перебора связанного потока ввода-вывода.
- *Реверсивный итератор* (reverse iterator). Перемещается назад, а не вперед. У всех библиотечных контейнеров, кроме `forward_list`, есть реверсивные итераторы.
- *Итератор перемещения* (move iterator). Итератор специального назначения; он перемещает элементы, а не копирует. Эти итераторы рассматриваются в разделе 13.6.2 (стр. 687).



10.4.1. Итераторы вставки

Адаптер вставки (inserter), или адаптер `inserter`, — это адаптер итератора (см. раздел 9.6, стр. 473), получающий контейнер и возвращающий итератор, позволяющий вставлять элементы в указанный контейнер. Присвоение значения при помощи итератора вставки приводит к вызову контейнерной функции, добавляющей элемент в определенную позицию заданного контейнера. Операторы, поддерживающие эти итераторы, приведены в табл. 10.2.

Таблица 10.2. Операторы итератора вставки

<code>it = t</code>	Вставляет значение <code>t</code> в позицию, обозначенную итератором <code>it</code> . В зависимости от вида итератора вставки и с учетом того, что он связан с контейнером <code>c</code> , вызывает функции <code>c.push_back(t)</code> , <code>c.push_front(t)</code> и <code>c.insert(t, p)</code> , где <code>p</code> — позиция итератора, заданная адаптеру вставки
<code>*it, ++it, it++</code>	Эти операторы существуют, но ничего не делают с итератором <code>it</code> . Каждый оператор возвращает итератор <code>it</code>

Существуют три вида адаптеров вставки, которые отличаются позицией добавляемых элементов.

- Адаптер `back_inserter` (см. раздел 10.2.2, стр. 489) создает итератор, использующий функцию `push_back()`.

- Адаптер `front_inserter` создает итератор, использующий функцию `push_front()`.
- Адаптер `inserter` создает итератор, использующий функцию `insert()`. Кроме имени контейнера, адаптеру `inserter` передают второй аргумент: итератор, указывающий позицию, перед которой должна начаться вставка.



ВНИМАНИЕ

Адаптер `front_inserter` можно использовать, *только* если у контейнера есть функция `push_front()`. Аналогично адаптер `back_inserter` можно использовать, *только* если у контейнера есть функция `push_back()`.

Важно понимать, что при вызове адаптера `inserter(c, iter)` возвращается итератор, который при использовании вставляет элементы перед элементом, первоначально обозначенным итератором `iter`. Таким образом, если `it` — итератор, созданный адаптером `inserter`, то присвоение `*it = val;` ведет себя, как следующий код:

```
it = c.insert(it, val); // it указывает на недавно добавленный элемент
++it; // инкремент it, чтобы он указывал на тот же элемент, что и прежде
```

Итератор, созданный адаптером `front_inserter`, ведет себя прямо противоположно итератору, созданному адаптером `inserter`. При использовании адаптера `front_inserter` элементы всегда вставляются перед текущим первым элементом контейнера. Даже если переданная адаптеру `inserter` позиция первоначально обозначает первый элемент, то, как только будет вставлен элемент перед этим элементом, он больше не будет элементом в начале контейнера:

```
list<int> lst = {1,2,3,4};
list<int> lst2, lst3; // пустой список
// после завершения копирования, lst2 содержит 4 3 2 1
copy(lst.cbegin(), lst.cend(), front_inserter(lst2));
// после завершения копирования, lst3 содержит 1 2 3 4
copy(lst.cbegin(), lst.cend(), inserter(lst3, lst3.begin()));
```

Когда происходит вызов `front_inserter(c)`, возвращается итератор вставки, который последовательно вызывает функцию `push_front()`. По мере вставки каждого элемента он становится новым первым элементом контейнера `c`. Следовательно, адаптер `front_inserter` возвращает итератор, который полностью изменяет порядок последовательности, в которую осуществляется вставка; адаптеры `inserter` и `back_inserter` так не поступают.

Упражнения раздела 10.4.1

Упражнение 10.26. Объясните различия между тремя итераторами вставки.

Упражнение 10.27. В дополнение к функции `unique()` (см. раздел 10.2.3, стр. 492) библиотека определяет функцию `unique_copy()`, получающую третий итератор, обозначающий назначение копирования уникальных элементов. Напишите программу, которая использует функцию `unique_copy()` для копирования уникальных элементов вектора в первоначально пустой список.

Упражнение 10.28. Скопируйте вектор, содержащий значения от 1 до 9, в три других контейнера. Используйте адаптеры `inserter`, `back_inserter` и `front_inserter` соответственно для добавления элементов в эти контейнеры. Предскажите вид результирующей последовательности в зависимости от вида адаптера вставки и проверьте свои предсказания на написанной программе.



10.4.2. Потокосные итераторы

Хотя типы `iostream` не относятся к контейнерам, есть итераторы, применимые к объектам типов ввода-вывода (см. раздел 8.1, стр. 400). Итератор `istream_iterator` (табл. 10.3) читает входной поток, а итератор `ostream_iterator` (табл. 10.4, стр. 519) пишет в поток вывода. Эти итераторы рассматривают свой поток как последовательность элементов определенного типа. Используя потоковый итератор, можно применять обобщенные алгоритмы для чтения или записи данных в объекты потоков.

Таблица 10.3. Операторы итератора `istream_iterator`

<code>istream_iterator<T> in(is);</code>	<code>in</code> читает значения типа <code>T</code> из входного потока <code>is</code>
<code>istream_iterator<T> end;</code>	Итератор после конца для итератора <code>istream_iterator</code> , читающего значения типа <code>T</code>
<code>in1 == in2</code>	<code>in1</code> и <code>in2</code> должны читать одинаковый тип. Они равны, если оба они конечные или оба связаны с тем же входным потоком
<code>in1 != in2</code>	
<code>*in</code>	Возвращает значение, прочитанное из потока
<code>in->mem</code>	Синоним для <code>(*in).mem</code>
<code>++in, in++</code>	Читает следующее значение из входного потока, используя оператор <code>>></code> для типа элемента. Как обычно, префиксная версия возвращает ссылку на итератор после инкремента. Постфиксная версия возвращает прежнее значение

Использование итератора `istream_iterator`

Когда создается потоковый итератор, необходимо определить тип объектов, которые итератор будет читать или записывать. Итератор `istream_iterator` использует оператор `>>` для чтения из потока. Поэтому тип, читаемый итератором `istream_iterator`, должен определять оператор ввода. При создании итератора `istream_iterator` следует связать с потоком. В качестве альтернативы итератор можно инициализировать значением по умолчанию. В результате будет создан итератор, который можно использовать как значение после конца.

```
istream_iterator<int> int_it(cin);    // читает целые числа из cin
istream_iterator<int> int_eof;       // конечное значение итератора
ifstream in("afile");
istream_iterator<string> str_it(in); // читает строки из "afile"
```

Для примера используем итератор `istream_iterator` для чтения со стандартного устройства ввода в вектор:

```
istream_iterator<int> in_iter(cin); // читает целые числа из cin
istream_iterator<int> eof;          // "'конечный'" итератор istream
while (in_iter != eof)              // пока есть что читать
    // постфиксный инкремент читает поток и возвращает прежнее значение
    // итератора. Обращение к значению этого итератора предоставляет
    // предыдущее значение, прочитанное из потока
    vec.push_back(*in_iter++);
```

Этот цикл читает целые числа из потока `cin`, сохраняя прочитанное в вектор `vec`. На каждой итерации цикл проверяет, не совпадает ли итератор `in_iter` со значением `eof`. Этот итератор был определен как пустой итератор `istream_iterator`, который используется как конечный итератор. Связанный с потоком итератор равен конечному итератору, только если связанный с ним поток достиг конца файла или произошла ошибка ввода-вывода.

Самая трудная часть этой программы — аргумент функции `push_back()`, который использует обращение к значению и постфиксные операторы инкремента. Это выражение работает точно так же, как и другие выражения, совмещающие обращение к значению с постфиксным инкрементом (см. раздел 4.5, стр. 204). Постфиксный инкремент переводит поток на чтение следующего значения, но возвращает прежнее значение итератора. Это прежнее значение содержит прежнее значение, прочитанное из потока. Для того чтобы получить это значение, осуществляется обращение к значению этого итератора.

Особенно полезно то, что эту программу можно переписать так:

```
istream_iterator<int> in_iter(cin), eof; // читает целые числа из cin
vector<int> vec(in_iter, eof);           // создает вектор vec из
                                         // диапазона итераторов
```

Здесь вектор `vec` создается из пары итераторов, которые обозначают диапазон элементов. Это итераторы `istream_iterator`, следовательно, диапазон

получается при чтении связанного потока. Этот конструктор читает поток `cin`, пока он не встретит конец файла, или ввод, тип которого отличается от `int`. Прочитанные элементы используются для создания вектора `vec`.

Использование потоковых итераторов с алгоритмами

Поскольку алгоритмы используют функции итераторов, а потоковые итераторы поддерживают по крайней мере некоторые функции итератора, потоковые итераторы можно использовать с некоторыми из алгоритмов. Какие именно алгоритмы применимы с потоковыми итераторами, рассматривается в разделе 10.5.1 (стр. 525). В качестве примера рассмотрим вызов функции `accumulate()` с парой итераторов `istream_iterator`:

```
istream_iterator<int> in(cin), eof;  
cout << accumulate(in, eof, 0) << endl;
```

Этот вызов создаст сумму значений, прочитанных со стандартного устройства ввода. Если ввод в этой программе будет таким:

```
23 109 45 89 6 34 12 90 34 23 56 23 8 89 23
```

то результат будет 664.

Итераторы `istream_iterator` позволяют использовать ленивое вычисление

То, что итератор `istream_iterator` связан с потоком, еще не гарантирует, что он начнет читать поток немедленно. Некоторые реализации разрешают задерживать чтение потока, пока итератор не будет использован. Гарантировано, что поток будет прочитан перед первым обращением к значению итератора. Для большинства программ не имеет никакого значения, будет ли чтение немедленным или отсроченным. Но если создается итератор `istream_iterator`, который удаляется без использования, или если необходима синхронизация чтения того же потока из двух разных объектов, то тогда придется позаботиться и о моменте чтения.

Использование итератора `ostream_iterator`

Итератор `ostream_iterator` может быть определен для любого типа, у которого есть оператор вывода (оператор `<<`). При создании итератора `ostream_iterator` можно (необязательно) предоставить второй аргумент, определяющий символьную строку, выводимую после каждого элемента. Это должна быть символьная строка в стиле C (т.е. строковый литерал или указатель на массив с нулевым символом в конце). Итератор `ostream_iterator` следует связать с определенным потоком. Не бывает пустого итератора `ostream_iterator` или такового после конца.

Таблица 10.4. Операторы итератора `ostream_iterator`

<code>ostream_iterator<T> out(os);</code>	out пишет значения типа T в поток вывода os
<code>ostream_iterator<T> out(os, d);</code>	out пишет значения типа T, сопровождаемые d в поток вывода os. d указывает на символьный массив с нулевым символом в конце
<code>Out = val</code>	Записывает val в поток вывода, с которым связан out, используя оператор <<. Тип val должен быть совместим с типом, который можно писать в out
<code>*out, ++out, out++</code>	Эти операторы существуют, но ничего не делают с out. Каждый оператор возвращает итератор out

Итератор `ostream_iterator` можно использовать для записи последовательности значений:

```
ostream_iterator<int> out_iter(cout, " ");
for (auto e : vec)
    *out_iter++ = e; // это присвоение запишет элемент в cout
cout << endl;
```

Эта программа запишет каждый элемент вектора `vec` в поток `cout`, сопровождая каждый элемент пробелом. При каждом присвоении значения итератора `out_iter` происходит запись.

Следует заметить, что при присвоении итератору `out_iter` можно пропустить обращение к значению и инкремент. Таким образом, этот цикл можно переписать так:

```
for (auto e : vec)
    out_iter = e; // это присвоение запишет элемент в cout
cout << endl;
```

Операторы `*` и `++` ничего не делают с итератором `ostream_iterator`, поэтому их пропуск никак не влияет на программу. Но предпочтительней писать цикл как в первом варианте. Он использует итератор единообразно с тем, как используются итераторы других типов. Этот цикл можно легко изменить так, чтобы он выполнялся итераторами других типов. Кроме того, поведение этого цикла понятней читателям нашего кода.

Чтобы не сочинять цикл самостоятельно, можно легко ввести элементы в вектор `vec` при помощи алгоритма `copy()`:

```
copy(vec.begin(), vec.end(), out_iter);
cout << endl;
```

Использование потоковых итераторов с типами класса

Итератор `istream_iterator` можно создать для любого типа, у которого есть оператор ввода (`>>`). Точно так же итератор `ostream_iterator` можно определить для любого типа, обладающего оператором вывода (`<<`). Поскольку у класса `Sales_item` есть оба оператора (ввода и вывода), итераторы ввода-вывода вполне можно использовать, чтобы переписать программу книжного магазина из раздела 1.6 (стр. 52):

```
istream_iterator<Sales_item> item_iter(cin), eof;
ostream_iterator<Sales_item> out_iter(cout, "\n");
// сохранить первую транзакцию в sum и читать следующую запись
Sales_item sum = *item_iter++;
while (item_iter != eof) {
    // если текущая транзакция (хранящаяся в item_iter) имеет тот же ISBN
    if (item_iter->isbn() == sum.isbn())
        sum += *item_iter++; // добавить ее к sum и читать следующую
                               // транзакцию
    else {
        out_iter = sum;      // вывести текущую сумму
        sum = *item_iter++;  // читать следующую транзакцию
    }
}
out_iter = sum; // не забыть вывести последний набор записей
```

Эта программа использует итератор `item_iter` для чтения транзакций `Sales_item` из потока `cin`. Она использует итератор `out_iter` для записи полученной суммы в поток `cout`, сопровождая каждый вывод символом новой строки. Определив итераторы, используем итератор `item_iter` для инициализации переменной `sum` значением первой транзакции:

```
// сохранить первую транзакцию в sum и читать следующую запись
Sales_item sum = *item_iter++;
```

Выражение осуществляет обращение к значению результата постфиксного инкремента итератора `item_iter`. Затем оно читает следующую транзакцию и инициализирует переменную `sum` значением, предварительно сохраненным в `item_iter`.

Цикл `while` выполняется до тех пор, пока поток `cin` не встретит конец файла. В цикле `while` осуществляется проверка, не относится ли содержимое переменной `sum` и только что прочитанная запись к той же книге. Если это так, то только что прочитанный объект класса `Sales_item` добавляется в переменную `sum`. Если ISBN отличаются, переменная `sum` присваивается итератору `out_iter`, который выводит текущее значение переменной `sum`, сопровождаемое символом новой строки. После вывода суммы для предыдущей книги переменной `sum` присваивается копия последней прочитанной транзакции и осуществляется инкремент итератора для чтения следующей транзакции. Цикл продолжается до конца файла или ошибки чтения. Перед завершением следует вывести значения по последней книге во вводе.

Упражнения раздела 10.4.2

Упражнение 10.29. Напишите программу, использующую потоковые итераторы для чтения текстового файла в вектор строк.

Упражнение 10.30. Используйте потоковые итераторы, а также функции `sort()` и `copy()` для чтения последовательности целых чисел со стандартного устройства ввода, их сортировки и последующего вывода на стандартное устройство вывода.

Упражнение 10.31. Измените программу из предыдущего упражнения так, чтобы она выводила только уникальные элементы. Программа должна использовать алгоритм `unique_copy()` (см. раздел 10.4.1, стр. 516).

Упражнение 10.32. Перепишите программу книжного магазина из раздела 1.6 (стр. 52). Используйте вектор для хранения транзакции и различные алгоритмы для обработки. Используйте алгоритм `sort()` с собственной функцией `compareIsbn()` из раздела 10.3.1 (стр. 496) для упорядочивания транзакций, а затем используйте алгоритмы `find()` и `accumulate()` для вычисления суммы.

Упражнение 10.33. Напишите программу, получающую имена входного и двух выходных файлов. Входной файл должен содержать целые числа. Используя итератор `istream_iterator`, прочитайте входной файл. Используя итератор `ostream_iterator`, запишите нечетные числа в первый выходной файл. За каждым значением должен следовать пробел. Во второй файл запишите четные числа. Каждое из этих значений должно быть помещено в отдельную строку.

10.4.3. Реверсивные итераторы

Реверсивный итератор (reverse iterator) перебирает контейнер в обратном направлении, т.е. от последнего элемента к первому. Реверсивный итератор инвертирует смысл инкремента (и декремента): оператор `++it` переводит реверсивный итератор на предыдущий элемент, а оператор `--it` — на следующий.

Реверсивные итераторы есть у всех контейнеров, кроме `forward_list`. Для получения реверсивного итератора используют функции-члены `rbegin()`, `rend()`, `crbegin()` и `crend()`. Они возвращают реверсивные итераторы на последний элемент в контейнере и на “следующий” (т.е. предыдущий) перед началом контейнера. Подобно обычным итераторам, существуют константные и неконстантные реверсивные итераторы.

Взаимное положение этих четырех итераторов на гипотетическом векторе `vec` представлено на рис. 10.1.

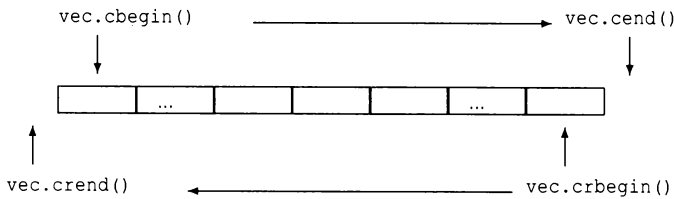


Рис. 10.1. Взаимное положение итераторов, возвращаемых функциями `begin()/end()` и `rbegin()/rend()`

Рассмотрим, например, следующий цикл, выводящий элементы вектора `vec` в обратном порядке:

```
vector<int> vec = {0,1,2,3,4,5,6,7,8,9};
// реверсивный итератор вектора (от конца к началу)
for (auto r_iter = vec.crbegin(); // связывает r_iter с последним
    // элементом
    r_iter != vec.crend(); // crend ссылается на 1 элемент
    // перед 1-м
    ++r_iter) // декремент итератора на один элемент
    cout << *r_iter << endl; // выводит 9, 8, 7, ... 0
```

Хотя смысл оператора декремента реверсивного итератора может показаться неправильным, этот оператор позволяет применять для обработки контейнера стандартные алгоритмы. Например, передав функции `sort()` два реверсивных итератора, вектор можно отсортировать в порядке убывания.

```
sort(vec.begin(), vec.end()); // сортирует вектор vec в "нормальном"
                             // порядке
// обратная сортировка: самый маленький элемент располагается
// в конце вектора vec
sort(vec.rbegin(), vec.rend());
```

Реверсивным итераторам необходим оператор декремента

Нет ничего удивительного в том, что реверсивный итератор можно создать только из такого класса итератора, для которого определены операторы `-` и `++`. В конце концов, задача реверсивного итератора заключается в переборе последовательности назад. Кроме контейнера `forward_list`, итераторы всех стандартных контейнеров поддерживают как инкремент, так и декремент. Однако потоковые итераторы к ним не относятся, поскольку невозможно перемещать поток в обратном направлении. Следовательно, создать из потокового итератора реверсивный итератор невозможно.



Отношения между реверсивными и другими итераторами

Предположим, что существует объект `line` класса `string` (строка), содержащий разделяемый запятыми список слов. Используя функцию `find()`, можно отобразить, например, первое слово строки `line`:

```
// найти первый элемент в списке, разделенном запятыми
auto comma = find(line.cbegin(), line.cend(), ',');
cout << string(line.cbegin(), comma) << endl;
```

Если в строке `line` есть запятая, итератор `comma` будет указывать на нее, в противном случае он будет равен итератору, возвращаемому функцией `line.cend()`. При выводе содержимого строки от позиции `line.cbegin()` до позиции `comma` будут отображены символы от начала до запятой или вся строка, если запятых в ней нет.

Но если понадобится последнее слово в списке, то вместо обычных можно использовать реверсивные итераторы:

```
// найти последний элемент в списке, разделенном запятыми
auto rcomma = find(line.crbegin(), line.crend(), ',');
```

Поскольку функции `find()` в качестве аргументов передаются результаты выполнения функций `crbegin()` и `crend()`, поиск начинается с последнего символа в строке `line` в обратном порядке. По завершении поиска, если запятая найдена, итератор `rcomma` будет указывать на последнюю запятую в строке, т.е. первую запятую с конца. Если запятой нет, итератор `rcomma` будет равен итератору, возвращаемому функцией `line.crend()`.

Весьма интересна та часть, в которой осуществляется вывод найденного слова. Попытка прямого вывода создает несколько странный результат:

```
// ошибка: создаст слово в обратном порядке
cout << string(line.crbegin(), rcomma) << endl;
```

Например, если введена строка **"FIRST,MIDDLE,LAST"**, будет получен результат **"TSAL"**!

Эта проблема проиллюстрирована на рис. 10.2. Здесь реверсивные итераторы используются для перебора строки в обратном порядке. Поэтому оператор вывода выводит строку `line` назад, начиная от `crbegin()`. Вместо этого следует выводить строку от `rcomma` и до конца. Но итератор `rcomma` нельзя использовать непосредственно, так как это реверсивный итератор, обеспечивающий перебор от конца к началу. Поэтому необходимо преобразовать его назад в обычный итератор, перебирающий строку вперед. Для преобразования итератора `rcomma` можно применить функцию-член `base()`, которой обладает каждый реверсивный итератор.

```
// ok: получить прямой итератор и читать до конца строки
cout << string(rcomma.base(), line.cend()) << endl;
```

С учетом того, что введены те же данные, в результате отобразится слово **"LAST"**, как и ожидалось.

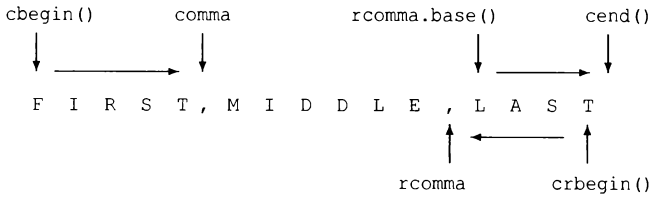


Рис. 10.2. Отношения между реверсивными и обычными итераторами

Объекты, представленные на рис. 10.2, наглядно иллюстрируют взаимоотношения между обычными и реверсивными итераторами. Например, итераторы `rcomma` и возвращаемый функцией `rcomma.base()` указывают на разные элементы, так же как и возвращаемые функциями `line.crbegin()` и `line.cend()`. Эти различия вполне обоснованны: они позволяют гарантировать возможность одинаковой обработки диапазона элементов при перемещении как вперед, так и назад.

С технической точки зрения отношения между обычными и реверсивными итераторами приспособлены к свойствам диапазона, включающего левый элемент (см. раздел 9.2.1, стр. 426). Дело в том, что `[line.crbegin(), rcomma)` и `[rcomma.base(), line.cend())` ссылаются на тот же элемент в строке `line`. Для этого `rcomma` и `rcomma.base()` должны возвращать соседние позиции, а не ту же позицию, как функции `crbegin()` и `cend()`.



Тот факт, что реверсивные итераторы предназначены для представления диапазонов и что эти диапазоны являются асимметричными, имеет важное последствие: при инициализации или присвоении реверсивному итератору простого итератора полученный в результате итератор не будет указывать на тот же элемент, что и исходный.

Упражнения раздела 10.4.3

Упражнение 10.34. Используйте итератор `reverse_iterator` для вывода содержимого вектора в обратном порядке.

Упражнение 10.35. Теперь отобразите элементы в обратном порядке, используя обычные итераторы.

Упражнение 10.36. Используйте функцию `find()` для поиска в списке целых чисел последнего элемента со значением 0.

Упражнение 10.37. С учетом того, что вектор содержит 10 элементов, скопируйте в список диапазон его элементов от позиции 3 до позиции 7 в обратном порядке.

10.5. Структура обобщенных алгоритмов

Фундаментальное свойство любого алгоритма — это список функциональных возможностей, которые он требует от своего итератора (итераторов). Некоторые алгоритмы, например `find()`, требуют только возможности получить доступ к элементу через итератор, прирастить итератор и сравнить два итератора на равенство. Другие, такие как `sort()`, требуют возможности читать, писать и произвольно обращаться к элементам. По своим функциональным возможностям, обязательным для алгоритмов, итераторы группируются в пять *категорий* (iterator categories), перечисленных в табл. 10.5. Каждый алгоритм определяет, итератор какого вида следует предоставить для каждого из его параметров.

Таблица 10.5. Категории итераторов

Итератор ввода	Обеспечивает чтение, но не запись; поддерживает только инкремент
Итератор вывода	Обеспечивает запись, но не чтение; поддерживает только инкремент
Прямой итератор	Обеспечивает чтение и запись; поддерживает только инкремент
Двунаправленный итератор	Обеспечивает чтение и запись; поддерживает инкремент и декремент
Итератор произвольного доступа	Обеспечивает чтение и запись; поддерживает все арифметические операции итераторов

Второй способ классификации алгоритмов (приведенный в начале этой главы) основан на том, читают ли они элементы, пишут или переупорядочивают их в последовательности. В приложении А все алгоритмы перечислены согласно этой классификации.

Алгоритмы имеют также ряд общих соглашений по передаче параметров и соглашений об именовании, рассматриваемых после категорий итераторов.

10.5.1. Пять категорий итераторов

Подобно контейнерам, для итераторов определен общий набор операций. Некоторые из них поддерживаются всеми итераторами, а другие — лишь некоторыми видами итераторов. Например, итератор `ostream_iterator` поддерживает только инкремент, обращение к значению и присвоение. Итераторы векторов, строк и двухсторонних очередей поддерживают эти операции, а также декремент, сравнение и арифметические операторы.

Таким образом, итераторы можно классифицировать на основании набора функций, которыми они обладают, а категории формируют своего рода иерархию. За исключением итераторов вывода, итераторы более высокой категории поддерживают все функции итераторов более низких категорий.

Стандарт определяет минимальную категорию для каждого параметра итератора обобщенных и числовых алгоритмов. Например, алгоритм `find()`, реализующий перебор последовательности только для чтения и в одном направлении, минимально требует только итератор ввода. Алгоритму `replace()` требуется два итератора, являющихся, по крайней мере, прямыми итераторами. Аналогично алгоритм `replace_copy()` требует прямые итераторы для своих первых двух итераторов. Его третий итератор, представляющий назначение, должен, по крайней мере, быть итератором вывода и т.д. Итератор для каждого параметра должен обладать не меньшим набором параметров, чем предусмотренный минимум. Передача итератора с меньшими возможностями недопустима.



Большинство компиляторов не заметит ошибки передачи алгоритму итератора неправильный категории.

Категории итераторов

Итератор ввода (input iterator) позволяет читать элементы контейнера, но записи не гарантирует. Итератор ввода обязательно должен поддерживать следующий минимум функций.

- Операторы равенства и неравенства (`==`, `!=`), используемые для сравнения двух итераторов.
- Префиксный и постфиксный инкременты (`++`), используемые для перемещения итератора.
- Оператор обращения к значению (`*`), позволяющий прочитать элемент. Оператор обращения к значению может быть применен только к операнду, расположенному справа от оператора присвоения.
- Оператор стрелки (`->`), равнозначный выражению `(*it).member`. То есть обращение к значению итератора и доступ к члену класса объекта.

Итераторы ввода могут быть использованы только последовательно. Гарантирована допустимость инкремента `*it++`, но приращение итератора ввода может сделать недопустимыми все другие итераторы в потоке. В результате нет никакой гарантии того, что можно сохранить состояние итератора ввода и исследовать элемент с его помощью. Поэтому итераторы ввода можно использовать только

для однопроходных алгоритмов. Алгоритмам `find()` и `accumulate()` требуются итераторы ввода, а итератору `istream_iterator` — итераторы ввода.

Итератор вывода (`output iterator`) можно рассматривать как итератор ввода, обладающий дополнительными функциональными возможностями. Итератор вывода применяется для записи в элемент, но чтения он не гарантирует. Для итераторов вывода обязательны следующие функции.

- Префиксный и постфиксный инкременты (`++`), используемые для перемещения итератора.
- Оператор обращения к значению (`*`) может быть применен только к операнду, расположенному слева от оператора присвоения. Присвоение при обращении к значению итератора вывода позволяет осуществить запись в элемент.

Значение итератору вывода можно присвоить только однажды. Подобно итераторам ввода, итераторы вывода можно использовать только для однопроходных алгоритмов. Итераторы, используемые как итераторы назначения, обычно являются итераторами вывода. Например, третий параметр алгоритма `copy()` является итератором вывода. Итератор `ostream_iterator` имеет тип итератора вывода.

- *Прямой итератор* (`forward iterator`) позволяет читать и записывать данные в последовательность. Они перемещаются по последовательности только в одном направлении. Прямые итераторы поддерживают все операции итераторов ввода и вывода. Кроме того, они позволяют читать и записывать значение в тот же элемент несколько раз. Поэтому сохраненное состояние прямого итератора можно использовать. Следовательно, алгоритмы, использующие прямые итераторы, могут осуществить несколько проходов через последовательность. Алгоритму `replace()` требуется прямой итератор; итераторы контейнера `forward_list` являются прямыми итераторами.
- *Двунаправленный итератор* (`bidirectional iterator`) позволяет читать и записывать данные в последовательность в обоих направлениях. Кроме всех функциональных возможностей прямого итератора, двунаправленный итератор поддерживает также префиксный и постфиксный декременты (`--`). Алгоритму `reverse()` требуется двунаправленный итератор. Все библиотечные контейнеры, кроме `forward_list`, предоставляют итераторы, соответствующие требованиям для двунаправленного итератора
- *Итератор прямого доступа* (`random-access iterator`) обеспечивает доступ к любой позиции последовательности в любой момент. Эти итераторы обладают всеми функциональными возможностями двунаправленных

итераторов. Кроме того, они поддерживают операции, приведенные в табл. 3.7 (стр. 159).

- Операторы сравнения $<$, $<=$, $>$ и $>=$, позволяющие сравнить относительные позиции двух итераторов.
- Операторы сложения и вычитания ($+$, $+=$, $-$ и $-=$), обеспечивающие арифметические действия между итератором и целочисленным значением. В результате получается итератор, перемещенный в контейнере вперед (или назад) на соответствующее количество элементов.
- Оператор вычитания ($-$), применяемый к двум итераторам, позволяет получить дистанцию между двумя итераторами.
- Оператор индексирования ($iter[n]$), равнозначный выражению $*(iter + n)$.

Итератор прямого доступа необходим алгоритму `sort()`. Итераторы контейнеров `array`, `deque`, `string` и `vector` являются итераторами прямого доступа, подобно указателям массива.

Упражнения раздела 10.5.1

Упражнение 10.38. Перечислите пять категорий итераторов и операции, которые каждый из них поддерживает.

Упражнение 10.39. Итератором какой категории обладает список? А вектор?

Упражнение 10.40. Итераторы какой категории нужны алгоритму `copy()`? А алгоритмам `reverse()` и `unique()`?



10.5.2. Параметрическая схема алгоритмов

Эта классификация алгоритмов основана на соглашениях об именах параметров. Понимание этих соглашений поможет в изучении новых алгоритмов: зная, что означает имя данного параметра, можно догадаться, какие операции выполняет соответствующий алгоритм. Большинство алгоритмов получают параметры в одной из четырех форм:

```
алг( beg, end, другие параметры );
алг( beg, end, dest, другие параметры );
алг( beg, end, beg2, другие параметры );
алг( beg, end, beg2, end2, другие параметры );
```

где *алг* — это имя алгоритма, а параметры *beg* и *end* обозначают исходный диапазон элементов, с которыми работает алгоритм. Хотя почти все алгоритмы получают исходный диапазон, присутствие других параметров зависит от выполняемых действий. Как правило, остальные параметры, *dest*, *beg2*

и `end2`, также являются итераторами. Кроме них, некоторые алгоритмы получают дополнительные параметры, не являющиеся итераторами.

Алгоритмы с одним итератором назначения

Параметр `dest` (*destination* — назначение) — это итератор, обозначающий получателя, используемого для хранения результата. Алгоритмы *подразумевают*, что способны безопасно записать в последовательность назначения столько элементов, сколько необходимо.



Алгоритмы, осуществляющие запись по итератору вывода, подразумевают, что получатель достаточно велик для содержания вывода.

Если `dest` является итератором контейнера, алгоритм записывает свой результат в уже существующие элементы контейнера. Как правило, итератор `dest` связан с итератором вставки (см. раздел 10.4.1, стр. 514) или итератором `ostream_iterator` (см. раздел 10.4.2, стр. 516). Итератор вставки добавляет новые элементы в контейнер, гарантируя, таким образом, достаточную емкость. Итератор `ostream_iterator` осуществляет запись в поток вывода, а следовательно, тоже не создает никаких проблем независимо от количества записываемых элементов.

Алгоритмы с двумя итераторами, указывающими исходную последовательность

Алгоритмы, получающие один параметр (`beg2`) или два параметра (`beg2` и `end2`), используют эти итераторы для обозначения второго исходного диапазона. Как правило, для выполнения необходимых действий эти алгоритмы используют элементы второго диапазона вместе с элементами исходного.

Когда алгоритм получает параметры `beg2` и `end2`, эти итераторы обозначают весь второй диапазон. Такой алгоритм получает два полностью определенных диапазона: исходный диапазон, обозначенный итераторами `[beg, end)`, а также второй, исходный диапазон, обозначенный итераторами `[beg2, end2)`.

Алгоритмы, получающие только итератор `beg2` (но не `end2`), рассматривают итератор `beg2` как указывающий на первый элемент во втором исходном диапазоне. Конец этого диапазона не определен. В этом случае алгоритмы *подразумевают*, что диапазон, начинающийся с элемента, указанного итератором `beg2`, имеет, по крайней мере, такой же размер, что и диапазон, обозначенный итераторами `beg` и `end`.



Алгоритмы, получающие один параметр `beg2`, *подразумевают*, что последовательность, начинающаяся с элемента, указанного итератором `beg2`, имеет такой же размер, как и диапазон, обозначенный итераторами `beg` и `end`.



10.5.3. Соглашения об именовании алгоритмов

Кроме соглашений об именовании параметров, алгоритмы также имеют набор однозначных соглашений об именовании перегруженных версий. Эти соглашения учитывают то, как предоставляется оператор, используемый вместо принятого по умолчанию оператора `<` или `==`, а также то, используется ли алгоритм для исходной последовательности или отдельного получателя.

Некоторые алгоритмы используют перегруженные версии для передачи предиката

Как правило, перегружаются алгоритмы, которые получают предикат для использования вместо оператора `<` или `==` и не получающие других аргументов. Одна версия функции использует для сравнения элементов оператор типа элемента, а вторая получает дополнительный параметр, являющийся предикатом, используемым вместо оператора `<` или `==`:

```
unique(beg, end);           // использует для сравнения элементов оператор ==
unique(beg, end, comp);     // использует для сравнения элементов
                             // предикат comp
```

Оба вызова переупорядочивают переданную последовательность, удаляя смежные повторяющиеся элементы. Первая версия для проверки на совпадение использует оператор `==` типа элемента, а вторая вызывает для этого предикат `comp`. Поскольку эти версии функции отличаются количеством аргументов, нет никакой неоднозначности (см. раздел 6.4, стр. 305) относительно версии вызываемой функции.

Алгоритмы с версиями `_if`

У алгоритмов, получающих значение элемента, обычно есть вторая (не перегруженная) версия, получающая предикат (см. раздел 10.3.1, стр. 494) вместо значения. Получающие предикат алгоритмы имеют суффикс `_if`:

```
find(beg, end, val);       // найти первый экземпляр val в исходном диапазоне
find_if(beg, end, pred);   // найти первый экземпляр, для
                             // которого pred возвращает true
```

Оба алгоритма находят в исходном диапазоне первый экземпляр заданного элемента. Алгоритм `find()` ищет указанное значение, а алгоритм `find_if()` — значение, для которого предикат `pred` возвратит значение, отличное от нуля.

Эти алгоритмы предоставляют вторую именованную версию, а не перегруженную, поскольку обе версии алгоритма получают то же количество аргументов. Перегрузка была бы неоднозначна и возможна лишь в некоторых случаях. Чтобы избежать любых неоднозначностей, библиотека предоставляет отдельные именованные версии этих алгоритмов.

Различия между копирующими и не копирующими версиями

По умолчанию переупорядочивающие элементы алгоритмы записывают результирующую последовательность в исходный диапазон. Эти алгоритмы предоставляют вторую версию, способную записывать результат по указанному назначению. Как уже упоминалось, пригодные для записи по назначению алгоритмы имеют в имени суффикс `_copy` (см. раздел 10.2.2, стр. 491):

```
reverse(beg, end); // обратить порядок элементов в исходном диапазоне
reverse_copy(beg, end, dest); // скопировать элементы по назначению в
                             // обратном порядке
```

Некоторые алгоритмы предоставляют и версии `_copy`, и `_if`. Эти версии получают и итератор назначения, и предикат:

```
// удаляет нечетные элементы из v1
remove_if(v1.begin(), v1.end(),
          [](int i) { return i % 2; });
// копирует только четные элементы из v1 в v2; v1 неизменен
remove_copy_if(v1.begin(), v1.end(), back_inserter(v2),
               [](int i) { return i % 2; });
```

Для определения нечетности элемента оба вызова используют лямбда-выражение (см. раздел 10.3.2, стр. 497). В первом случае нечетные элементы удаляются из самой исходной последовательности. Во втором нечетные (четные) элементы копируются из исходного диапазона в вектор `v2`.

Упражнения раздела 10.5.3

Упражнение 10.41. Исходя только из имен алгоритмов и их аргументов, опишите действия, выполняемые каждым из следующих библиотечных алгоритмов:

```
replace(beg, end, old_val, new_val);
replace_if(beg, end, pred, new_val);
replace_copy(beg, end, dest, old_val, new_val);
replace_copy_if(beg, end, dest, pred, new_val);
```

10.6. Алгоритмы, специфические для контейнеров

В отличие от других контейнеров, контейнеры `list` и `forward_list` определяют несколько алгоритмов в качестве членов. В частности, тип `list` определяют собственные версии алгоритмов `sort()`, `merge()`, `remove()`, `reverse()` и `unique()`. Обобщенная версия алгоритма `sort()` требует итераторов произ-

вольного доступа. В результате она не может использоваться с контейнерами `list` и `forward_list`, поскольку эти типы предоставляют двунаправленные и прямые итераторы соответственно.

Обобщенные версии других алгоритмов, определяемых типом `list`, вполне применимы со списками, но ценой производительности. Эти алгоритмы меняют элементы в исходной последовательности. Список может “поменять” свои элементы, поменяв ссылки на элементы, а не перемещая значения этих элементов. В результате специфические для списка версии этих алгоритмов могут обеспечить намного лучшую производительность, чем соответствующие обобщенные версии.

Эти специфические для списка функции приведены в табл. 10.6. В ней нет обобщенных алгоритмов, которые получают соответствующие итераторы и выполняются одинаково эффективно как для других контейнеров, так и для контейнеров `list` и `forward_list`.



Предпочтительней использовать алгоритмы-члены классов `list` и `forward_list`, а не их обобщенные версии.

Таблица 10.6. Алгоритмы-члены классов `list` и `forward_list`

Эти функции возвращают <code>void</code> .	
<code>lst.merge(lst2)</code> <code>lst.merge(lst2, comp)</code>	Объединяет элементы списков <code>lst2</code> и <code>lst</code> . Оба списка должны быть отсортированы. Элементы из списка <code>lst2</code> удаляются, и после объединения список <code>lst2</code> оказывается пустым. Возвращает тип <code>void</code> . В первой версии используется оператор <code><</code> , а во второй — указанная функция сравнения
<code>lst.remove(val)</code> <code>lst.remove_if(pred)</code>	При помощи функции <code>lst.erase()</code> удаляет каждый элемент, значение которого равно переданному значению, или для которого указанный унарный предикат возвращает значение, отличное от нуля
<code>lst.reverse()</code>	Меняет порядок элементов списка <code>lst</code> на обратный
<code>lst.sort()</code> <code>lst.sort(comp)</code>	Сортирует элементы списка <code>lst</code> , используя оператор <code><</code> или другой заданный оператор сравнения
<code>lst.unique()</code> <code>lst.unique(pred)</code>	При помощи функции <code>lst.erase()</code> удаляет расположенные рядом элементы с одинаковыми значениями. Вторая версия использует заданный бинарный предикат



Алгоритм-член `splice()`

Типы списков определяют также алгоритм `splice()`, описанный в табл. 10.7. Этот алгоритм специфичен для списочных структур данных. Следовательно, обобщенная версия этого алгоритма не нужна.

Таблица 10.7. Аргументы алгоритма-члена `splice()` классов `list` и `forward_list`

<code>lst.splice(аргументы)</code> или <code>flst.splice_after(аргументы)</code>	
<code>(p, lst2)</code>	<code>p</code> — итератор на элемент списка <code>lst</code> или итератор перед элементом списка <code>flst</code> . Перемещает все элементы из списка <code>lst2</code> в список <code>lst</code> непосредственно перед позицией <code>p</code> или непосредственно после в списке <code>flst</code> . Удаляет элементы из списка <code>lst2</code> . Список <code>lst2</code> должен иметь тот же тип, что и <code>lst</code> (или <code>flst</code>), и не может быть тем же списком
<code>(p, lst2, p2)</code>	<code>p2</code> — допустимый итератор в списке <code>lst2</code> . Перемещает элемент, обозначенный итератором <code>p2</code> , в список <code>lst</code> или элемент после обозначенного итератором <code>p2</code> в списке <code>flst</code> . Список <code>lst2</code> может быть тем же списком, что и <code>lst</code> или <code>flst</code>
<code>(p, lst2, b, e)</code>	<code>b</code> и <code>e</code> обозначают допустимый диапазон в списке <code>lst2</code> . Перемещает элементы в заданный диапазон из списка <code>lst2</code> . Списки <code>lst2</code> и <code>lst</code> (или <code>flst</code>) могут быть тем же списком, но итератор <code>p</code> не должен указывать на элемент в заданном диапазоне

Специфические для списка функции, изменяющие контейнер

Большинство специфических для списков алгоритмов подобны (но не идентичны) их обобщенным аналогам. Но кардинально важное различие между специфическими и обобщенными версиями в том, что специфические версии изменяют базовый контейнер. Например, специфическая версия алгоритма `remove()` удаляет указанные элементы. Специфическая версия алгоритма `unique()` удаляет второй и последующий дубликаты элемента.

Аналогично алгоритмы `merge()` и `splice()` деструктивны к своим аргументам. Например, обобщенная версия функции `merge()` запишет объединенную последовательность по заданному итератору назначения; две исходных последовательности останутся неизменными. Специфическая для списка функция `merge()` разрушит заданный список — элементы будут удаляться из списка аргумента по мере их объединения в объект, для которого был вызван аргумент `merge()`. После объединения элементы из обоих списков продолжают существовать, но принадлежат уже одному списку.

Упражнения раздела 10.6

Упражнение 10.42. Переделайте программу, устранявшую повторяющиеся слова, написанную в разделе 10.2.3 (стр. 491), так, чтобы использовался список, а не вектор.

Резюме

Стандартная библиотека определяет порядка ста независимых от типа алгоритмов, работающих с последовательностями. Последовательности могут быть элементами контейнера библиотечного типа, встроенного массива или созданного (например, при чтении или записи в поток). Алгоритмы достигают независимости от типа, работая только с итераторами. Большинство алгоритмов получает их как первые два аргумента, обозначающие диапазон элементов. К дополнительным аргументам может относиться итератор вывода, обозначающий получателя, или другой итератор, или пара итераторов, обозначающая вторую исходную последовательность.

Итераторы подразделяются на пять категорий в зависимости от поддерживаемых ими функциональных возможностей. Категории итераторов: ввода, вывода, прямой, двунаправленный и произвольного доступа. Итератор относится к определенной категории, если он поддерживает функциональные возможности, обязательные для итератора данной категории.

Подобно категоризации итераторов по их функциональным возможностям, параметры итераторов для алгоритмов классифицируются по функциональным возможностям требуемых для них итераторов. Алгоритмам, которые только читают из последовательности, достаточно итератора ввода. Алгоритмам, которые пишут по итератору назначения, требуются итераторы вывода и т.д.

Алгоритмы *никогда* непосредственно не изменяют размер последовательности, с которой они работают. Они могут скопировать элементы из одной позиции в другую, но не могут самостоятельно добавить или удалить элемент.

Хотя алгоритмы не могут добавлять элементы в последовательность, итератор вставки может создавать их. Итератор вставки связан с контейнером. При присвоении значения типа элемента контейнера итератору вставки он добавляет данный элемент в контейнер.

Контейнеры `forward_list` и `list` определяют собственные версии некоторых из обобщенных алгоритмов. В отличие от обобщенных алгоритмов, эти специфические версии изменяют переданные им списки.

Термины

Адаптер `back_inserter`. Адаптер итератора, который, получив ссылку на контейнер, создает итератор вставки, использующий функцию `push_back()` для добавления элементов в указанный контейнер.

Адаптер `front_inserter`. Адаптер итератора, который, получив ссылку на контейнер, создает итератор вставки, использующий функцию `push_front()` для добавления элементов в начало указанного контейнера.

Адаптер `inserter`. Адаптер итератора, который, получив итератор и ссылку на контейнер, создает итератор вставки, используемый функцией `insert()` для добавления элементов непосредственно перед элементом, указанным данным итератором.

Бинарный предикат (binary predicate). Предикат с двумя параметрами.

Вызываемый объект (callable object). Объект, способный быть левым операндом оператора вызова. К вызываемым объектам относятся указатели на функции, лямбда-выражения и объекты классов, определяющих перегруженные версии оператора вызова.

Двунаправленный итератор (bidirectional iterator). Поддерживает те же операции, что и прямые итераторы, плюс способность использовать оператор `--` для перемещения по последовательности назад.

Итератор `istream_iterator`. Поточковый итератор, обеспечивающий чтение из потока ввода.

Итератор `ostream_iterator`. Поточковый итератор, обеспечивающий запись в поток вывода.

Итератор ввода (input iterator). Итератор, позволяющий читать, но не записывать элементы.

Итератор вставки (insert iterator). Итератор, использующий функции контейнера для добавления элементов в данный контейнер.

Итератор вывода (output iterator). Итератор, позволяющий записывать, но не обязательно читать элементы.

Итератор перемещения (move iterator). Итератор, позволяющий перемещать элементы, а не копировать их. Итераторы перемещения рассматриваются в главе 13.

Итератор прямого доступа (random-access iterator). Поддерживает те же операции, что и двунаправленный итератор, плюс способность использовать операторы сравнения для выяснения позиций двух итераторов относительно друг друга, а также возможность осуществлять с итераторами арифметические действия, обеспечивая таким образом произвольный доступ к элементам.

Категории итераторов (iterator categories). Концептуальная организация итераторов на основании поддерживаемых ими операций. Категории итераторов составляют иерархию, в которой более мощные итераторы представляют те же операции, что и менее мощные. Пока итератор обеспечивает, по крайней мере, достаточный уровень операций, он вполне применим. Например, некоторым алгоритмам требуются только итераторы ввода. Такие алгоритмы могут быть применены к любому другому итератору, который обладает возможностями, не ниже, чем у итератора вывода. Алгоритмы, которым необходимы итераторы прямого доступа, применимы только для тех итераторов, которые поддерживают операции прямого доступа.

Лямбда-выражение (lambda expression). Вызываемый блок кода. Лямбды немного похожи на безымянные встраиваемые функции. Они начинаются со списка захвата, позволяющего лямбда-выражению получать доступ к переменным в содержащей функции. Подобно функции, имеет список параметров (возможно пустой), тип возвращаемого значения и тело функции. У лямбда-выражения может отсутствовать тип возвращаемого значения. Если тело функции представляет собой одиночный оператор `return`, тип возвращаемого значения выводится из типа возвращаемого объекта. В противном случае типом пропущенного возвращаемого значения по умолчанию принимается `void`.

Обобщенный алгоритм (generic algorithm). Алгоритм, не зависящий от типа контейнера.

Потоковый итератор (stream iterator). Итератор, который может быть связан с потоком.

Предикат (predicate). Функция, которая возвращает значение типа `bool` (логическое) или допускающее преобразование в него. Зачастую используется обобщенными алгоритмами для проверки элементов. Используемые библиотекой предикаты являются либо унарными (получающими один аргумент), либо бинарными (получающими два аргумента).

Прямой итератор (forward iterator). Итератор, позволяющий читать и записывать элементы, но не поддерживающий оператор `--`.

Реверсивный итератор (reverse iterator). Итератор, позволяющий перемещаться по последовательности назад. У этих итераторов операторы `++` и `--` имеют противоположный смысл.

Список захвата (capture list). Часть лямбда-выражения, определяющая переменные из окружающего контекста, к которым может обращаться лямбда-выражение.

Унарный предикат (unary predicate). Предикат с одним параметром.

Функция `bind()`. Библиотечная функция, связывающая один или несколько аргументов с вызываемым выражением. Функция `bind()` определена в заголовке `functional`.

Функция `cref()`. Библиотечная функция, возвращающая копируемый объект, содержащий ссылку на константный объект типа, не допускающего копирования.

Функция `ref()`. Библиотечная функция, создающая копируемый объект из ссылки на объект типа, не допускающего копирования.